

Centro de Investigación y de Estudios Avanzados
del Instituto Politécnico Nacional
Unidad Guadalajara

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis que presenta:

Miguel Angel Salazar Martinez

para obtener el grado de:

Maestro en Ciencias

en la especialidad de:

Ingeniería Eléctrica

Director de Tesis

Dr. Mario Angel Siller González Pico

Guadalajara, Jalisco Agosto de 2026

Auto-diseño de algoritmos en el marco de Sistemas de Auto-Ingeniería (SES)

Tesis de Maestría en Ciencias Ingeniería Eléctrica

Por:

Miguel Angel Salazar Martinez

Ingeniero en Sistemas Computaciones
Instituto Tecnológico de Ocotlán 2019-2023

Becario de SECIHTI, expediente no. 245700008

Director de Tesis

Dr. Mario Angel Siller González Pico

Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional
Unidad Guadalajara, Agosto de 2026

Resumen

Abstract

Dedicatoria

A mi madre por ser mi ejemplo a seguir. Agradezco infinitamente tu amor y apoyo incondicional. A mi hermana Saidy y mi hermano Diego, por impulsarme a seguir adelante y ser mi espacio seguro. A mi abuela por enseñarme lo que es ser un buen ser humano. A la vida, por permitirme llegar tan lejos.

Agradecimientos

A todos los que me apoyaron en el camino y me ayudaron en esta travesía del posgrado.
A mi asesor de tesis, el Dr. Mario Siller, por todo el apoyo durante el desarrollo del plan de estudios.
A mi querida familia, a mi madre, a mi padre, a mis hermanos, a mis abuelos, a mis tíos y primos que tanto me han apoyado.
Al grupo de redes de CINVESTAV Guadalajara, que más que grupo, es una familia.
A los doctores del área de computación, por compartir su conocimiento, consejos y, sobre todo, por cultivar la tenacidad. Esa que no aparece en la rubrica, pero que resulta indispensable para llegar hasta aquí.
A todos mis amigos (incluyendo los compis y roomies) especialmente a los miembros del laboratorio de computación, por esas largas charlas, orientación, momentos de calidez y de recreación.
A todos mis maestros, quienes me educaron en toda mi vida académica y nunca desistieron al enseñarme.
Entre ellos destaco a la profesora Aurora Berenice Navarro Núñez, por darme el apoyo que necesite al momento de ingresar a este posgrado.
A todos aquellos que creyeron en mí.
Finalmente, quiero agradecer a la SECIHTI por el apoyo económico brindado, con el cual se logró concluir esta maestría.
Muchas Gracias a todos.

Quiero aclarar que no nombré a todos los involucrados, no por falta de gratitud, sino porque en extensión, los agradecimientos serian inmensos.

Índice general

Resumen I

Abstract II

Dedicatoria III

Agradecimientos IV

1. Introducción 1

1.1. Planteamiento del Problema . . . 2
1.2. Motivación . . . 3
1.3. Objetivos . . . 4
1.3.1. Objetivo General . . . 4
1.3.2. Objetivos Específicos . . . 4
1.4. Hipótesis . . . 4
1.5. Estructura del documento de la Tesis . . . 5

2. Marco Teórico 6

2.1. Ingeniería de Software y el ciclo de vida de los sistemas . . . 6
2.2. Sistemas de Auto-Ingeniería . . . 7
2.2.1. Fases de Operación y Mantenimiento (O & M) y Evolución . . . 8
2.2.2. Actividades de Self-O & M y Self-Evolution . . . 8
2.2.3. Punto de Quiebre / Breaking Point . . . 8
2.2.4. Elasticidad del Sistema / System Elasticity . . . 8
2.2.5. Metamorfosis / Metamorphosis . . . 8
2.2.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM) . . . 9
2.2.7. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería . . . 10

2.3. Actividades de diseño . . . 10
2.3.1. Diseño de Arquitectura . . . 11
2.3.2. Diseño de Interfaces . . . 12
2.3.3. Diseño de Componentes . . . 12
2.3.4. Diseño de Base de datos . . . 12
2.3.5. Diseño de Algoritmos . . . 13
2.4. Sistemas Multiagentes (MAS) . . . 13
2.4.1. Fundamentos y arquitecturas de los Sistemas Multiagentes . . . 13
2.4.2. Paradigmas de coordinación y comunicación . . . 14
2.4.3. Comunicación por pizarra o tablero (Blackboard Communication) . . . 15
2.5. Modelos grandes de lenguaje (LLM) . . . 15
2.5.1. Principios de inferencia y generación de texto en modelos de lenguaje 16
2.5.2. Capacidades de los LLMs en la comprensión, diseño y generación de código . . . 16
2.5.3. Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista . . . 17
2.6. Árbol de búsqueda de Monte-Carlo (MCTS) . . . 17
2.7. Ontología como representación de conocimiento . . . 18
2.7.1. Definición y estructura de las ontologías computacionales (clases, atributos, relaciones) . . . 18
2.7.2. Lenguajes de modelado ontológico . . . 19
2.7.3. Modelado semántico para la taxonomía y clasificación de problemas algorítmicos . . . 20

3. Estado del Arte **22**
3.1. Conceptualización del Auto-Diseño . . . 22
3.2. Automatización del Diseño . . . 23
3.2.1. Auto-diseño basado en Inteligencia Artificial y Aprendizaje . . . 23
3.2.2. Autodiseño bio-inspirado, de reúso y búsqueda . . . 24
3.2.3. Autodiseño basado en modelado de agentes . . . 24
3.3. Conceptos Emergentes y Mecanismos de Control en Autodiseño . . . 25
3.4. Comparación de Trabajos . . . 25

4. Propuesta **26**
4.1. Metodología . . . 26

4.1.1. Revisión Sistemática de literatura . . . 26
4.1.1.1. Formulación preguntas de investigación . . . 26
4.1.1.2. Estrategia de búsqueda . . . 27
4.1.1.3. Criterios de Inclusión y de exclusión . . . 31
4.1.1.4. Proceso de selección de estudios . . . 33
4.1.1.5. Extracción de datos . . . 35
4.1.1.6. Resultados . . . 36
4.1.1.7. Discusión . . . 36
4.1.2. Propuesta de un Sistema Multi-Agentes (MAS) para satisfacer el auto-diseño . . . 36
4.1.3. Desarrollo de la metodología Prometheus . . . 37
4.1.3.1. Especificación del sistema . . . 38
4.1.3.2. Diseño arquitectónico . . . 39
4.1.3.3. Diseño detallado . . . 40
4.1.4. Análisis y validación del modelo de agentes . . . 41
4.2. Propuesta de marco conceptual y arquitectura . . . 42
4.2.1. Definición conceptual y formal . . . 43
4.2.2. Propuesta detallada . . . 43
4.2.2.1. Agente de arquitectura . . . 43
4.2.2.2. Agente de componentes e interfaces . . . 44
4.2.2.3. Agente de estructuras de datos . . . 44
4.2.2.4. Agente de algoritmos . . . 45
4.2.3. Mecanismo de coordinación y consenso . . . 45
4.2.4. Resumen conceptual . . . 46

5. Experimentación y Análisis de Resultados **48**
5.1. Análisis de selección automática de algoritmos . . . 48
5.1.1. Análisis de la arquitectura: auto-diseño . . . 48
5.2. Experimentación . . . 48
5.2.1. Caso de estudio: nuevo problema, nueva solución . . . 48
5.2.2. Experimento: Identificación del Problema . . . 48
5.2.3. Experimento: Identificación del Algoritmo . . . 48

6. Conclusiones y Trabajo Futuro **49**
6.1. Conclusiones . . . 49

6.2. Objetivos logrados 49
6.3. Trabajo a futuro 49

Referencias Bibliográficas **50**

A. Código Fuente Relevante **56**

Índice de figuras

2.1. Diagrama del proceso de diseño inspirado en el diagrama 2.5 de Ingenieria de software de Sommerville.(Diagrama elaborado en Draw.io) 11

4.1. Revisión del flujo de trabajo basado en PRISMA.(Diagrama elaborado en Draw.io) 34

4.2. Proceso de selección de artículos con resultados cuantitativos por etapa de cribado. (Diagrama elaborado en Draw.io) 35

Índice de tablas

3.1. Matriz comparativa de enfoques de automatización y auto-diseño de software. 25

4.1. Actividades de diseño y sus correspondientes artefactos de diseño. 27
4.2. Ejemplo de una consulta de búsqueda completa (Scopus) 29
4.3. Resumen de las consultas en Scopus (Obviando los filtros temporales y de dominio para mantener la brevedad). 30
4.4. Resumen de los criterios de inclusión y exclusión 32

Capítulo 1

Introducción

El desarrollo de sistemas de software modernos requiere la ejecución coordinada de múltiples actividades de ingeniería de software, desde el análisis de requerimientos, diseño, implementación, pruebas, operación y mantenimiento hasta el desecho final de los sistemas [1].

A lo largo de las últimas décadas, se ha evidenciado que las actividades de mantenimiento, actualización y evolución de artefactos de software representan una etapa con alto coste en términos de capital humano, tiempo de desarrollo y recursos financieros [2][3]. Con el objetivo de eliminar la dependencia de la intervención humana y reducir el tiempo de desarrollo se origina la teoría de los Sistemas de Auto-Ingeniería (*Self-Engineering Systems* - SES), la cual introduce un marco conceptual y una meta-arquitectura que habilitan a los sistemas ejecutar modificaciones autónomas sobre sí mismos, abriendo la posibilidad de realizar procesos de auto-evolución o metamorfosis [4]. **El propósito fundamental de un sistema SES es gestionar y ejecutar su propio proceso de ingeniería más allá de su ciclo de vida convencional, prescindiendo de la intervención humana.**[5]

Una de las actividades fundamentales y de mayor impacto en la ingeniería de software es la actividad de diseño [6]. Esta actividad es crítica debido a que es la encargada de traducir los requerimientos funcionales y no funcionales en especificaciones técnicas detalladas para la posterior implementación y ejecución del sistema. Para auto-diseñar en el contexto de los sistemas SES[5], se persigue la automatización de las actividades que conforman el proceso tradicional de diseño, es decir, **dotar al sistema de capacidades de auto-diseño** implica que la plataforma no solo reaccione a fallos, sino que sea capaz de reconfigurar conceptualmente su propia lógica interna de manera válida, segura y coherente con las decisiones arquitectónicas previas.

Diversos trabajos han explorado la automatización de actividades específicas del diseño de software; sin embargo, la evidencia reportada en la literatura sugiere que estas iniciativas suelen enfocarse en etapas particulares del proceso de diseño y rara vez abordan su integración de manera integral, o permanecen como demostraciones conceptuales que no cubren de manera integral todas las actividades del diseño. Esto se analiza a profundidad en el capítulo 3.

En respuesta a esta problemática, **esta investigación propone una meta-arquitectura basada en un sistema multiagente para la generación autónoma de artefactos de diseño**. Una meta-arquitectura establece el marco de referencia y las reglas de composición que permiten abstraer, generalizar y coordinar múltiples configuraciones de orden inferior bajo un modelo unificado [7]. La propuesta se compone de cuatro agentes especializados responsables de las actividades de diseño arquitectónico, diseño de componentes e interfaces, diseño de estructuras de datos y diseño algorítmico. En conjunto, dichos agentes colaboran para construir una especificación progresivamente refinada a partir de los requerimientos del sistema.

La presente investigación define una meta-arquitectura que contempla las actividades de diseño descritas por Sommerville Et. Al. [6]. No obstante, como prueba de concepto y mecanismo de validación, el desarrollo e implementación se concentran en el agente responsable del diseño algorítmico.

Las principales contribuciones de esta investigación son las siguientes:

1. Una caracterización del estado del arte sobre diseño automatizado de software mediante una revisión sistemática de literatura.

2. Una meta-arquitectura basada en sistemas multiagente para la generación autónoma y coordinada de artefactos de diseño de software.

3. Un modelo de agente de diseño algorítmico fundamentado en representación ontológica del conocimiento y asociación problema-solución para la selección automática de algorítmicas.

1.1. Planteamiento del Problema

Durante el desarrollo, operación y mantenimiento de un sistema de software, este experimenta cambios en sus requerimientos que pueden dar lugar a la generación de nuevos artefactos de diseño o a la modificación de los ya existentes; este proceso se conoce como

actividad de diseño [6]. La actividad de diseño abarca desde la definición de la arquitectura del sistema y la especificación de sus componentes hasta la selección de estructuras de datos, el diseño de algoritmos y otros artefactos con distinto nivel de detalle, como diagramas y casos de uso [8]. Bajo la teoría de los Sistemas Evolutivos de Software (SES) [5], se busca que estas actividades puedan realizarse sin intervención humana. Sin embargo, aún no existe un mecanismo que automatice la actividad de diseño dentro de dicha teoría. A partir de este planteamiento surgen los siguientes cuestionamientos:

1. ¿Es posible dotar a un sistema de un mecanismo de diseño completamente automatizado?

2. Ante cambios en los requerimientos de un sistema en operación, ¿cómo pueden generarse nuevos artefactos de diseño o modificarse los existentes y, en caso necesario, sustituirse de manera consistente?

3. Cuando se produce un nuevo sistema mediante un proceso de metamorfosis, ¿en qué medida es posible reutilizar los artefactos y el conocimiento previamente generados?

Los artefactos resultantes deben permitir documentar y comunicar las decisiones de diseño, facilitar su validación y mantener la trazabilidad entre los requerimientos, la arquitectura, los componentes y la implementación.

1.2. Motivación

La presente investigación surge de la necesidad de avanzar hacia sistemas de software capaces de participar activamente en su propia evolución mediante la generación y adaptación de artefactos de diseño. En entornos dinámicos y sujetos a cambios constantes, la dependencia de la intervención humana para rediseñar soluciones limita la capacidad de respuesta y adaptación de los sistemas. En este contexto, la automatización de las actividades de diseño de software representa un paso fundamental para habilitar mecanismos de razonamiento y toma de decisiones sobre la estructura y comportamiento de los sistemas. La integración de agentes especializados y mecanismos de representación explícita del conocimiento permite establecer las bases para que futuras generaciones de software puedan modificar y justificar sus decisiones de diseño de manera consistente. Finalmente, esta investigación busca contribuir al desarrollo de capacidades de autodiseño que funcionen como habilitador para la evolución y metamorfosis de sistemas SES [4], buscando erradicar la dependencia del mantenimiento manual.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar una meta-arquitectura basada en agentes para el diseño automatizado de software, capaz de abstraer y coordinar las distintas actividades de diseño con el fin de facilitar la evolución autónoma de artefactos de software ante cambios en los requerimientos.

1.3.2. Objetivos Específicos

- Sintetizar el estado del arte del diseño automatizado de software y la selección algorítmica autónoma mediante una Revisión Sistemática de la Literatura (RSL o SLR del ingles Sistematic Literature Review), con el fin de identificar tendencias, limitaciones y oportunidades de investigación.
- Diseñar una meta-arquitectura basada en sistemas multiagente que modele y coordine las actividades de diseño de software, definiendo mecanismos de interacción entre los distintos niveles de diseño y preservando la coherencia entre los artefactos generados.
- Desarrollar un agente de diseño algorítmico fundamentado en representación ontológica del conocimiento, capaz de establecer asociaciones problema-solución para la selección autónoma de alternativas algorítmicas.
- Validar la viabilidad de la meta-arquitectura propuesta mediante la implementación y evaluación del agente de diseño algorítmico en escenarios experimentales controlados, analizando su capacidad para adaptarse a cambios en los requerimientos y mantener la consistencia de los artefactos de diseño generados.

1.4. Hipótesis

Es posible gestionar de manera autónoma la generación y adaptación de artefactos de diseño de software ante cambios en los requerimientos mediante una meta-arquitectura basada en agentes especializados, capaz de coordinar distintas actividades de diseño y seleccionar alternativas de solución consistentes con las restricciones y decisiones previamente establecidas.

1.5. Estructura del documento de la Tesis

La estructura de este documento se organiza en seis capítulos interconectados que guían el desarrollo de la investigación.

El **Capítulo 1** establece la base del estudio mediante la introducción del problema, la motivación, los objetivos, las hipótesis y el contexto general. El **Capítulo 2** expone los fundamentos teóricos indispensables para la comprensión del trabajo, abordando conceptualizaciones clave sobre Sistemas de Auto-Ingeniería (SES, por sus siglas en inglés), automatización, diseño de software y selección algorítmica. Posteriormente, el **Capítulo 3** ofrece una revisión y análisis crítico del estado del arte, identificando las tendencias actuales, los enfoques metodológicos existentes, así como sus limitaciones y brechas de conocimiento. En el **Capítulo 4** se detalla la propuesta central de esta investigación, especificando su metodología, conceptualización, arquitectura y los mecanismos de selección automática diseñados. La validación experimental de dicha propuesta se describe en el **Capítulo 5** a través de casos de estudio, la simulación de escenarios y el análisis riguroso de los resultados obtenidos. Finalmente, el **Capítulo 6** cierra el documento exponiendo las conclusiones generales, las contribuciones principales de la investigación, sus limitaciones intrínsecas y las directrices sugeridas para el trabajo futuro.

Capítulo 2

Marco Teórico

El Marco Teórico constituye el cimiento conceptual sobre el cual se erige esta investigación. Su propósito fundamental es articular las teorías, paradigmas y modelos previos que otorgan validez científica a la problemática planteada, sirviendo como mapa orientador para el desarrollo de la propuesta.

2.1. Ingeniería de Software y el ciclo de vida de los sistemas

La Ingeniería de Software se establece como una disciplina fundamental dentro de las ciencias computacionales, cuyo propósito principal es la aplicación de principios, métodos y herramientas sistemáticas para el desarrollo, operación y mantenimiento de software de alta calidad. No existe una única definición ni caracterización de este proceso de ingeniería, pero entre los trabajos que mejor la explican se encuentran [1], [9], esta disciplina abarca no solo los aspectos técnicos de la construcción de programas, sino también la gestión de proyectos, el análisis de requerimientos y la garantía de calidad. Por su parte, la define como un enfoque ingenieril que busca resolver problemas complejos de desarrollo mediante la aplicación de procesos estructurados y estandarizados.

En el corazón de la Ingeniería de Software se encuentra el Ciclo de Vida del Desarrollo de Sistemas (SDLC, por sus siglas en inglés Systems Development Life Cycle). Este modelo conceptual describe las fases necesarias para llevar un sistema informático desde su concepción inicial, pasando por su diseño y construcción, hasta su implementación, operación y eventual retirada [10].

Tradicionalmente, el ciclo de vida se estructura en etapas secuenciales bien definidas:

1. Análisis de Requisitos: Identificación y documentación de las necesidades del usuario y las restricciones del sistema.

2. Diseño del Sistema: Traducción de los requisitos en una arquitectura de software y especificaciones detalladas de componentes, bases de datos e interfaces.

3. Implementación (Codificación): Construcción del código fuente utilizando los lenguajes y herramientas de programación adecuados.

4. Pruebas (Validación y Verificación): Detección sistemática de errores, asegurando que el software cumpla con los requerimientos funcionales y no funcionales especificados.

5. Despliegue e Implementación: Puesta en marcha del sistema en el entorno de producción de los usuarios finales.

6. Mantenimiento: Evolución continua del software para corregir fallos, mejorar el rendimiento o adaptarlo a nuevos entornos tecnológicos.

7. Retiro o reemplazo<: Como se dispone del sistema y sus respectivas gestiones al termino se su vida o cuando entre otros sistema a relevarlo [1]. Dependiendo de la naturaleza del proyecto de investigación, estas fases pueden adoptarse mediante enfoques predictivos tradicionales (como el modelo en cascada) o metodologías ágiles e iterativas, las cuales permiten una mayor flexibilidad ante los cambios constantes en los requerimientos del entorno [1], [10].

2.2. Sistemas de Auto-Ingeniería

En el trabajo de Perez-Carrillo Et. Al. [4] se busca que los mismos sistemas no terminen el ciclo de su vida en desecho, si no, que sin intervención humana continúen su operación bajo el concepto de elasticidad del sistema o que sufran una metamorfosis dando pie a un nuevo sistema, esta perspectiva se conoce como el marco de los sistemas de auto-ingeniería. Los SES contemplan un proceso muy similar a la ingeniería de software tradicional, es decir, las actividades como lo son ingeniería de requerimientos, diseño, implementación, pruebas, etc.[6] tienen lugar en este marco de trabajo. Este trabajo unicamente esta enfocado en el diseño del software automatizado, basado en lineamientos presentes en [1]. Por lo tanto el foco de este manuscrito esta puesto en la producción de artefactos de diseño, que fungen como planos del sistema antes de implementación en código.

La teoría SES introduce conceptos que permiten identificar fases y actividades dentro de la vida de estos sistemas [5]:

2.2.1. Fases de Operación y Mantenimiento (O & M) y Evolución

A diferencia de la literatura tradicional que debate si el mantenimiento es un subconjunto de la evolución o viceversa, el marco SES [5] establece las fases de Operación y Mantenimiento (O&M) y Evolución como fases independientes.

- Fase de O&M: Se ejecutan mantenimientos correctivos, preventivos, adaptativos, aditivos y de mejora.
- Evolución por Mantenimiento: Se genera cuando los mantenimientos adaptativos, aditivos o perfectivos provocan cambios en los requerimientos.
- Evolución por Cambio de Problema: Ocurre cuando cambia el problema que el sistema busca resolver, siempre y cuando la arquitectura actual sea capaz de soportar dichos cambios.

2.2.2. Actividades de Self-O & M y Self-Evolution

Son las actividades fundamentales que el sistema ejecuta de manera autónoma tras su despliegue dentro de las fases de O&M y Evolución, respectivamente, de allí el nombre de self, es decir desarrollado por si mismo.

2.2.3. Punto de Quiebre / Breaking Point

Representa el umbral en el cual la arquitectura original del sistema ya no puede satisfacer los nuevos requerimientos. Al alcanzar el **Breaking Point**, se suspenden las actividades de Self-O&M y Self-Evolution, desencadenando el inicio de un nuevo proceso e inicio de ciclo de vida de ingeniería presentado como **metamorfosis** [5].

2.2.4. Elasticidad del Sistema / System Elasticity

Fundamentada en los conceptos de evolucionabilidad (evolvability) y resiliencia. Se define como la capacidad del sistema para asimilar requerimientos nuevos o modificados sin la necesidad de construir un nuevo sistema.

2.2.5. Metamorfosis / Metamorphosis

Es la fase fundamental del paradigma SES, definida como la **trascendencia del sistema original (Sistema A) hacia un nuevo sistema (Sistema B)** mediante un nuevo

proceso de Auto-Ingeniería tras alcanzar un punto de ruptura. Involucra la ejecución coordinada de las actividades SES *Self-*: Self-Requirements, Self-Design, Self-Implementation, Self-Testing* y Self-Deployment.*

Se desencadena bajo dos condiciones principales:

- Cuando se requiere un cambio de arquitectura a nivel conceptual (el problema o dominio de aplicación cambió y la arquitectura actual no puede resolverlo).
- Cuando los cambios de requerimientos impactan la arquitectura lógica y esta es incapaz de acomodarlos.

A diferencia de la ingeniería tradicional donde un arquitecto humano toma la decisión, en el marco SES el propio sistema toma la decisión de manera autónoma.

2.2.6. Gestión de Cambios de Requerimientos / Requirements Change Management (RCM)

La estructura de requerimientos funcionales se divide en dos componentes principales, como se menciona a continuación. Se ha determinado que la funcionalidad se manifiesta a través de las acciones principales, expresadas en verbos, y la configuración. (parámetros de tiempo, modo o condición). En lo que respecta a los requerimientos no funcionales, las métricas y los parámetros se tratan como componentes de configuración. El impacto se distribuye en función del componente. En el caso de que el cambio se produzca únicamente en la configuración, el sistema lo asimila dentro de sus fases de O&M (Mantenimiento y Operación) y evolución. En el caso de que el cambio suponga una afectación a la funcionalidad o bien genere un impacto en la arquitectura que la actual no soporte, se provoca una metamorfosis. El procedimiento RCM, adaptado para SES (basado en [6]), se presenta de la siguiente manera:

- *Especificación del Cambio*: Recepción y validación de la solicitud de cambio.
- *Análisis del Cambio*: Evaluación de la elasticidad del sistema para determinar si el cambio produce una nueva versión o si se alcanzó el Breaking Point.
- *Implementación del Cambio*: Si la arquitectura lo soporta, el sistema evoluciona a una nueva versión; si no lo soporta, se inicia un nuevo proceso de Auto-Ingeniería dando lugar a un nuevo sistema (metamorfosis).

En el presente estudio se aborda la especificación del cambio, un aspecto de vital importancia en el ámbito de la gestión de procesos administrativos. En este sentido, se ha observado que la recepción y validación de la solicitud de cambio constituye un elemento fundamental en el proceso de implementación de modificaciones en los sistemas organizativos. Así mismo el análisis del cambio, para lo cual se ha llevado a cabo una evaluación de la elasticidad del sistema. Dicha evaluación tiene por objetivo determinar si el cambio produce una nueva versión o si se ha alcanzado el punto de inflexión. En el ámbito de la tecnología y la innovación, la implementación del cambio se erige como un factor determinante en la evolución de los sistemas de software. En este sentido, si la arquitectura del sistema es capaz de soportar las modificaciones, entonces se procede a la actualización a una nueva versión. Por el contrario, si la arquitectura no es capaz de soportar las modificaciones, se inicia un nuevo proceso de ingeniería automática que da lugar a la creación de un nuevo sistema, lo que se conoce como metamorfosis.

2.2.7. Meta-Arquitectura del Marco de Sistemas de Auto-Ingeniería

Un Sistema de Auto-Ingeniería (*SES*) se define por su capacidad para gestionar y llevar a cabo de manera autónoma su propio proceso de ingeniería a lo largo de su ciclo de vida y durante la trascendencia del mismo [5]. El núcleo operacional de este paradigma se fundamenta en la Meta-Arquitectura de Referencia, denominada *Ether*, una abstracción estructural que permanece persistente por encima de la arquitectura central del sistema y se conserva a lo largo de las transiciones entre sucesivas versiones del mismo (incluyendo nuevos sistemas) [5]. Cabe señalar que las capacidades de gestión, evolución y metamorfosis abordadas en las secciones previas son orquestadas por el propio *Ether*.

2.3. Actividades de diseño

Siguiendo el modelo general de diseño propuesto por Sommerville en las dos ultimas ediciones de "Software Engineering"[6], [9], se contemplan las actividades de Diseño Arquitectónico, Diseño de Interfaces, Diseño de Componentes y Diseño de Bases de Datos.

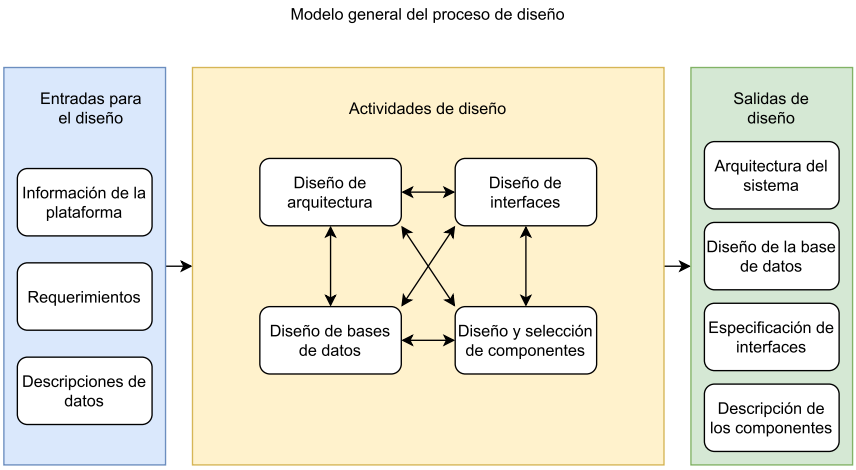


Figura 2.1: Diagrama del proceso de diseño inspirado en el diagrama 2.5 de Ingenieria de software de Sommerville.(Diagrama elaborado en Draw.io)

Adicionalmente, se incluye de forma explícita la actividad de Diseño de Algoritmos como parte del diseño detallado de componentes. Esta decisión se fundamenta en: La necesidad de especificar con precisión la lógica de procesamiento cuando los algoritmos presentan alta complejidad computacional o requisitos no funcionales críticos (rendimiento, tiempo real, etc.). La recomendación de estándares y guías de referencia como lo son el "Software Design Descriptions" (SDD) y la "Guide to the Software Engineering Body of Knowledge" [1], [11] que reconocen el diseño de algoritmos como una actividad diferenciada dentro del diseño detallado. La separación de preocupaciones (separation of concerns), que facilita la verificación, el análisis de complejidad y la posterior implementación.

2.3.1. Diseño de Arquitectura

Según Sommerville [6], el diseño arquitectónico consiste en identificar la estructura general del sistema, los componentes principales (también denominados sub-sistemas o módulos), sus relaciones y la forma en que se distribuyen. Esta actividad produce una visión de alto nivel que define cómo debe organizarse el sistema para satisfacer los requisitos funcionales y no funcionales. El SWEBOK [1] y el estándar IEEE 1016 **1016** refuerzan esta idea al considerar la arquitectura como la organización fundamental de los elementos de software y sus relaciones.

2.3.2. Diseño de Interfaces

Esta actividad define de manera precisa e inequívoca las interfaces entre los componentes del sistema. Sommerville Et. Al. [9] destaca que una vez especificadas las interfaces, los componentes pueden diseñarse y desarrollarse de forma concurrente, ya que cada uno solo necesita conocer el contrato de comunicación (operaciones, parámetros y comportamientos esperados). Esto favorece un bajo acoplamiento y una alta cohesión. El estandar IEEE 1016 [11] incluye el punto de vista de la interfaz (Interface Viewpoint) como una de las vistas recomendadas para documentar estos contratos.

2.3.3. Diseño de Componentes

En esta etapa se especifica con mayor precisión cómo operará cada uno de los componentes identificados durante el diseño arquitectónico. El diseño de componentes puede adoptar diferentes niveles de detalle según las necesidades del proyecto: desde una descripción funcional sencilla que indique qué debe hacer el componente, hasta la adaptación de un componente reutilizable ya existente, o bien la elaboración de un modelo de diseño detallado que defina su estructura interna, comportamiento y relaciones [9]. El SWEBOK sitúa esta actividad dentro del detailed design (diseño detallado), cuyo propósito fundamental es describir cada componente con el nivel de detalle suficiente para que pueda ser construido de manera inequívoca por los desarrolladores. Esto implica definir no solo las responsabilidades del componente, sino también sus interfaces internas, las estructuras de datos que manipula, los algoritmos principales que ejecuta y las restricciones de rendimiento o de calidad que debe satisfacer [1].

2.3.4. Diseño de Base de datos

El diseño de bases de datos consiste en definir las estructuras de datos del sistema y la forma en que éstas se representarán en una base de datos. Esta actividad abarca tanto el modelado lógico y físico de la base de datos (tablas, relaciones, índices, restricciones de integridad, etc.) como el diseño de las estructuras de datos en memoria que utilizarán los componentes y algoritmos del software (listas, árboles, tablas hash, grafos, colas, etc.)[9]. Existe una relación estrecha y bidireccional entre ambos aspectos: las necesidades de información y los patrones de acceso definidos en la base de datos influyen directamente en las estructuras de datos que se emplean en la aplicación, y a su vez, los requisitos de rendimiento y manipulación de datos de los algoritmos condicionan el diseño del esquema de la

base de datos. El SDD [11] refuerza esta visión a través del Punto de vista de informacion (Information Viewpoint), donde se especifican los elementos de diseño relacionados con la información: las entidades de diseño (data items, data types and classes, data stores y access mechanisms), las relaciones de diseño (asociacion, usos, implementaciones), los atributos de datos junto con sus restricciones y relaciones estáticas, así como los atributos de diseño de persistencia y propiedades de calidad.

2.3.5. Diseño de Algoritmos

Esta actividad forma parte del diseño detallado de componentes y se enfoca en la definición de los algoritmos que implementarán la lógica de procesamiento de cada componente. Su objetivo es seleccionar o diseñar soluciones algorítmicas que sean correctas, eficientes y adecuadas a los requisitos de rendimiento, tiempo de respuesta y uso de recursos del sistema. Aunque Sommerville [9] la integra dentro del diseño de componentes, estándares como el IEEE 1016 [11] (a través del Algorithm Viewpoint) y el SWEBOK [1]reconocen la importancia de especificar explícitamente los algoritmos cuando su complejidad o criticidad lo justifica. En este trabajo se trata como una actividad diferenciada para permitir un análisis riguroso de corrección, complejidad computacional y optimización.

2.4. Sistemas Multiagentes (MAS)

Los Sistemas Multiagentes (MAS, por sus siglas en inglés) constituyen una subdisciplina fundamental de la Inteligencia Artificial Distribuida (DAI). Se definen como redes de entidades computacionales autónomas (agentes) que interactúan dentro de un entorno dinámico compartiendo conocimiento y recursos para resolver problemas complejos que exceden las capacidades individuales de un solo agente. A diferencia del paradigma de resolución de problemas distribuida (DPS) convencional, los MAS destacan por su flexibilidad, capacidad de cómputo paralelo, escalabilidad modular y tolerancia a fallos mediante una degradación suave del sistema en caso de avería de alguno de sus componentes.[12]

2.4.1. Fundamentos y arquitecturas de los Sistemas Multiagentes

Un agente se conceptualiza como una entidad autónoma o hilo de ejecución concurrente que percibe su entorno a través de sensores y actúa sobre él mediante actuadores. Sus características primordiales abarcan la autonomía en la toma de decisiones, la reactividad

ante cambios del entorno, la proactividad orientada a metas y la capacidad de interacción social con otros agentes [12].

En cuanto a la **arquitectura interna (micro-arquitectura)**, los agentes se clasifican principalmente en:

- **Homogéneos:** Todos los agentes comparten la misma arquitectura interna, capacidad sensorial, estados locales y mecanismos de inferencia, diferenciándose únicamente en su ubicación física y en los datos recibidos del entorno.
- **Heterogéneos:** Los agentes difieren en sus estructuras, funcionalidades y objetivos locales, lo que permite modelar escenarios y dinámicas del mundo real con mayor fidelidad.

Asimismo, según sus modelos cognitivos y de control, los agentes abarcan arquitecturas reactivas, basadas en reglas, modelos deliberativos BDI (*Belief-Desire-Intention*), esquemas adaptativos basados en aprendizaje o enfoques híbridos que integran capas de control en tiempo real con módulos de inferencia superior.

En el nivel de la **organización global (macro-arquitectura)**, la literatura identifica diversos patrones organizacionales:

- **Jerárquica:** Estructura en árbol donde las directivas de control y supervisión fluyen desde niveles superiores a inferiores, mientras que los datos procesados ascienden en la jerarquía.
- **Holónica:** Organización basada en estructuras estables y fractales denominadas holones y superholones, donde subagentes vinculados por compromisos eligen un agente cabeza (*Head Agent*) para interactuar con el entorno exterior.
- **Coaliciones:** Agrupaciones dinámicas y temporales de agentes orientadas a maximizar la utilidad o el rendimiento en tareas específicas, disolviéndose una vez alcanzado el objetivo.
- **Equipos:** Estructuras cooperativas donde los agentes colaboran colectivamente para optimizar el desempeño global del grupo como un conjunto articulado.

2.4.2. Paradigmas de coordinación y comunicación

La interacción en un MAS es indispensable para prevenir conflictos por recursos distribuidos, evitar situaciones de caos y optimizar los objetivos del sistema. La comunicación

entre agentes se formaliza mediante Lenguajes de Comunicación de Agentes (ACL), como FIPA-ACL (Foundation for Intelligent Physical Agents - Agent Communication Language) o KQML (Knowledge Query and Manipulation Language), los cuales requieren una ontología compartida y un formato de protocolo estandarizado.

La **coordinación** dentro del sistema se articula a través de tres paradigmas principales:

- **Basada en Protocolos:** Emplea mecanismos de negociación distribuida como el *Contract Net Protocol* (CNP), donde los agentes administradores (Managers) subastan contratos a agentes ejecutores (Contractors) mediante licitaciones.
- **Basada en Grafos de Coordinación:** Descompone el problema global en subjuegos locales y calcula acciones conjuntas óptimas mediante la combinación lineal de pagos a través de algoritmos como la eliminación de variables o *max-plus*.
- **Basada en Modelos de Creencias:** Cada agente mantiene un modelo interno del comportamiento de sus vecinos, facilitando la toma de decisiones ágil en entornos dinámicos con restricciones temporales estrictas.

2.4.3. Comunicación por pizarra o tablero (Blackboard Communication)

A diferencia del paso directo de mensajes (*message passing*), la comunicación mediante **pizarra o tablero (*Blackboard*)** constituye un esquema centralizado y desacoplado donde un grupo de agentes comparte un repositorio común de información.

En este modelo, el *Blackboard* actúa como un almacenamiento compartido para datos de diseño y conocimiento de control. El acceso a esta memoria compartida se gestiona mediante una interfaz de red o *control shell*, encargada de notificar a los agentes cuando existe información relevante disponible. Aunque esta arquitectura simplifica la coordinación de grupos de agentes remotos, su principal limitación radica en que la pizarra representa un punto único de fallo (*single point of failure*), por lo que una avería central puede inhabilitar al grupo completo.

2.5. Modelos grandes de lenguaje (LLM)

Los modelos grandes de lenguaje (LLM, por sus siglas en inglés) son arquitecturas avanzadas basadas principalmente en la red neuronal Transformer que procesan y generan

lenguaje natural mediante un entrenamiento auto-supervisado masivo sobre extensos corpus de texto [13], [14], [15]. A través de esta etapa de pre-entrenamiento, el modelo comprime conocimiento lingüístico y conceptual de amplio dominio, lo que le permite identificar patrones semánticos y estructurales complejos de manera implícita. Como resultado, los LLM actúan como herramientas de propósito general capaces de resolver directamente tareas de redacción, análisis de texto, traducción y síntesis de información sin requerir entrenamiento específico para cada aplicación [13], [14], [15].

2.5.1. Principios de inferencia y generación de texto en modelos de lenguaje

La inferencia en los LLM se fundamenta en un proceso autorregresivo de predicción del siguiente token, condicionando probabilísticamente cada palabra o subpalabra generada al contexto previo y al *prompt* de entrada [14]. Esta arquitectura habilita la adaptación del comportamiento del modelo mediante el aprendizaje en contexto (*In-Context Learning* o ICL) y el seguimiento de instrucciones, respondiendo conforme a las restricciones y formatos especificados [15]. Sin embargo, desde una perspectiva operacional, los LLM se comportan como sistemas de çaja negra", donde la precisión y el estilo de la respuesta varían sustancialmente entre modelos en función de la arquitectura de la red, la escala y diversidad de los datos de entrenamiento, la ventana de contexto y la calidad del *prompt* [14], [15], [16].

2.5.2. Capacidades de los LLMs en la comprensión, diseño y generación de código

En la ingeniería de software y el desarrollo de sistemas, las capacidades de síntesis y comprensión de código de los LLM se derivan de la inclusión de vastos repositorios de software y documentación técnica durante su pre-entrenamiento [14]. La exposición a esta información especializada capacita al modelo para reconocer dependencias de largo alcance, sintaxis estructurada y patrones lógicos de programación [15]. Durante la fase de diseño de software, el modelo puede evaluar las especificaciones proporcionadas en el contexto y recomendar patrones de diseño adecuados, por ejemplo evaluar la arquitectura de un sistema y sugerir un enfoque Modelo-Vista-Controlador (MVC) al contrastar los requerimientos recibidos con las abstracciones aprendidas en su entrenamiento [13], [14]. Asimismo, los LLM aceleran el ciclo de vida del desarrollo mediante la generación

automática de código, la refactorización, el depurado y la creación de scripts de prueba automatizados [13], [14].

2.5.3. Limitaciones actuales: Alucinaciones, ventanas de contexto y deficiencias en el razonamiento algorítmico determinista

A pesar de sus amplias capacidades, los LLM presentan importantes limitaciones técnicas e inherentes [14], [16]. En primer lugar, el fenómeno de la alucinación ocurre cuando el modelo genera información inconsistente o no verificable respecto a las fuentes factuales existentes, debido a imprecisiones en los datos de entrenamiento, sesgos en las estrategias de muestreo estocástico y la naturaleza probabilística del objetivo de predicción de texto [13], [14]. En segundo lugar, las restricciones en la ventana de contexto condicionan el volumen de información procesable simultáneamente; aunque el texto se segmenta en tokens (unidades discretas de representación), secuencias de entrada extensas provocan degradación en la atención sobre detalles intermedios, fenómeno conocido como *lost in the middle* [15]. Por último, existen marcadas deficiencias en el razonamiento algorítmico determinista y la ejecución simbólica [16]. Incluso los modelos de razonamiento avanzado (*Large Reasoning Models* o LRM) diseñados para generar cadenas de pensamiento (*Chain-of-Thought*), sufren una caída drástica de precisión hasta el colapso total al incrementar la complejidad compositiva del problema [16]. Esto demuestra que los modelos no ejecutan algoritmos explícitos de manera rigurosa ni se benefician de la inclusión del algoritmo en el prompt, sino que dependen de la imitación probabilística de patrones aprendidos [16].

2.6. Árbol de búsqueda de Monte-Carlo (MCTS)

Un Monte Carlo Tree Search, (MCTS) es un algoritmo de búsqueda y toma de decisiones que construye un árbol de forma incremental mediante simulaciones estocásticas para evaluar opciones y elegir la acción más prometedora, cada nodo es un estado o configuraciones de un problema y las aristas son transiciones (acciones) de nodo a nodo [17].

El ciclo iterativo consta de cuatro fases: selección, expansión, simulación y propagación hacia atrás (retropropagacion).

- **Selección**: Se navega por el árbol desde la raíz hasta un nodo hoja, eligiendo nodos "buenos", estos nodos se etiquetan así al equilibrar dinámicamente dos objetivos: la *explotación* (favorecer estados con un historial de alto rendimiento o recompensa

acumulada) y la *exploración* (visitar estados poco explorados para reducir la incertidumbre y evitar mínimos locales) [17].

- **Expansión**: A partir del nodo hoja seleccionado, se añade uno o más nodos hijos para incorporar nuevas acciones (permitidas y viables) del espacio de búsqueda que aún no han sido exploradas en el árbol [17].
- **Simulación**: Se realiza un recorrido (frecuentemente aleatorio o guiado por heurísticas) desde el nodo recién expandido hasta alcanzar un estado terminal, lo que permite obtener una estimación empírica del valor del estado [17].
- **Retropropagacion**: El resultado de la simulación se propaga de forma ascendente desde el nodo evaluado hasta la raíz, actualizando las estadísticas de todos los nodos visitados en esa iteración (incrementando su contador de visitas y ajustando su valor acumulado o promedio) [17].

2.7. Ontología como representación de conocimiento

Una ontología es una especificación formal y explícita de una conceptualización compartida de un dominio [18], [19]. En el ámbito de las ciencias de la computación y la representación del conocimiento, las ontologías proporcionan un vocabulario estructurado que hace explícito el significado de los datos y las relaciones entre entidades [20]. Cumplen un doble propósito: un propósito **representacional**, enfocado en la enumeración declarativa de los hechos del dominio, y un propósito **computacional**, orientado a la derivación de inferencias lógicas a partir de los hechos representados. Esta capacidad inferencial permite que sistemas de software e inteligencia artificial procesen, compartan, reutilicen y comprendan datos de forma autónoma, sin requerir intervención humana previa ni conocimientos precodificados.

2.7.1. Definición y estructura de las ontologías computacionales (clases, atributos, relaciones)

Una ontología computacional organiza el conocimiento de un dominio a través de varios componentes lógicos interrelacionados:

- **Clases o Conceptos**: Representan las categorías generales o conjuntos de objetos dentro del dominio de discurso. Se estructuran mediante jerarquías de subclases,

donde los conceptos más específicos heredan las propiedades y características de las categorías más generales [18].

- **Relaciones o Propiedades de Objeto**: Definen los vínculos y asociaciones entre diferentes clases o individuos. Permiten especificar cómo interactúan las entidades e incluyen restricciones de dominio (qué conceptos pueden originar la relación) y de rango (qué conceptos pueden recibirla), además de características lógicas como transitividad, simetría o relaciones inversas [20].

- **Atributos o Propiedades de Datos**: Asocian a los individuos o instancias con valores de datos concretos y literales, tales como números enteros, cadenas de texto, fechas o valores booleanos.

- **Individuos o Instancias**: Corresponden a los objetos o entidades concretos y particulares que existen dentro del dominio y que pertenecen a una o más clases específicas [18].

- **Axiomas y Restricciones**: Reglas y afirmaciones lógicas que definen las condiciones necesarias y suficientes para la pertenencia a un concepto, restringen la interpretación del vocabulario y garantizan la consistencia lógica del sistema para fundamentar el razonamiento automático [18], [20].

2.7.2. Lenguajes de modelado ontológico

Los lenguajes de modelado ontológico han evolucionado con el objetivo de equilibrar la riqueza expresiva en la representación del conocimiento con la eficiencia computacional durante el proceso de inferencia automática [20]. Entre las tecnologías y estándares principales destacan:

- **RDF y RDFS**: El Marco de Descripción de Recursos (*Resource Description Framework*, RDF) proporciona una estructura básica basada en triples formados por sujeto, predicado y objeto para declarar datos en la Web. Por su parte, *RDF Schema* (RDFS) extiende este marco ofreciendo un vocabulario para organizar clases en jerarquías y definir el dominio y el rango de las propiedades [20].

- **Lógicas de Descripción**: Constituyen una familia de formalismos lógicos derivados de la lógica de primer orden que proveen una semántica formal rigurosa basada

en teoría de modelos, garantizando procedimientos de decisión decidibles para la deducción automática [20].

- **OWL (Web Ontology Language)**: Es el estándar del W3C diseñado para la representación de ontologías en la Web Semántica, construido sobre RDF/RDFS y fundamentado en las Lógicas de Descripción [20]. OWL amplía la capacidad expresiva de RDFS al permitir combinaciones lógicas de clases (unión, intersección y complemento), la declaración de clases disjuntas, restricciones locales sobre propiedades y restricciones de cardinalidad. OWL se divide en tres sublenguajes o perfiles adaptados a distintos requerimientos:

  1. **OWL Lite**: Ofrece una sintaxis reducida y menor complejidad computacional, orientada a usuarios que requieren jerarquías simples y restricciones básicas con un razonamiento rápido.
  2. **OWL DL**: Basado en lógicas de descripción expresivas, garantiza la decidibilidad del razonamiento (asegurando que los algoritmos de inferencia siempre terminan en un tiempo finito) mientras mantiene una elevada capacidad de representación formal.
  3. **OWL Full**: Ofrece máxima compatibilidad semántica con RDF y permite el metamodelado (como tratar clases como si fuesen individuos), aunque pierde la garantía de decidibilidad computacional.

2.7.3. Modelado semántico para la taxonomía y clasificación de problemas algorítmicos

El modelado semántico resulta fundamental para estructurar la caracterización de problemas de decisión y guiar la selección adaptativa de algoritmos en entornos computacionales dinámicos [21]. El trabajo de [22] introduce la taxonomía **TDMIoT** (*Taxonomy for Decision-Making in IoT Systems*), la cual conceptualiza el proceso de decisión como una asociación estructurada entre un problema de decisión y una solución algorítmica. TDMIoT organiza el dominio a través de siete dimensiones clave organizadas en dos espacios:

- **Espacio del Problema**: Se define mediante el *Objetivo* principal (pronóstico, optimización, planificación, resolución de conflictos o predicción esta última subclasificada en estimación, regresión y clasificación), el tipo de *Procesamiento* (centralizado o descentralizado), la *Capa de procesamiento* (percepción, middleware o aplicación),

el tipo de *Interacción* (del sistema o humana) y la fuente de *Incertidumbre* (de medición, de parámetros o de modelo).

- **Espacio de la Solución**: Se caracteriza mediante la *Representación del conocimiento* (basada en datos, simbólica, matemática o híbrida) y el *Enfoque algorítmico* (guiado por criterios, guiado por datos, guiado por utilidad o guiado por probabilidad).

Sobre la base conceptual de esta taxonomía, [21] propone la ontología **O-DMIoT** (*Ontology for Decision-Making in the IoT*), fundamentada en la ontología superior UFO (*Unified Foundational Ontology*) y desarrollada mediante la metodología de ingeniería ontológica SABiO. O-DMIoT formaliza el conocimiento organizándolo en capas lógicas interconectadas:

- **Capa de Requerimientos**: Caracteriza los problemas de decisión e independiza la definición del problema de la estrategia algorítmica, formalizando especificaciones de entrada, salida, parámetros y criterios de rendimiento.
- **Capa de Diseño**: Especifica las estrategias algorítmicas, supuestos, complejidad y restricciones teóricas del algoritmo sin vincularse al entorno de ejecución.
- **Capa de Ejecución**: Operativiza el algoritmo instanciándolo como un servicio desplegado dentro de la infraestructura tecnológica.
- **Rol de Evaluación**: Actúa de manera transversal asociando métricas de Calidad de Servicio (QoS), Calidad de Experiencia (QoE) y características intrínsecas del algoritmo (complejidad temporal y espacial) para evaluar el desempeño.

Este marco semántico permite abordar la selección de algoritmos como un proceso de planificación secuencial guiado por razonamiento ontológico y aprendizaje automático, posibilitando la reconfiguración adaptativa de servicios en tiempo de ejecución [21].

Capítulo 3

Estado del Arte

Este capítulo examina el estado del arte relacionado con el diseño automatizado de software y el concepto de *Self-Design* (diseño automatizado). Dado el carácter emergente de esta línea de investigación, la literatura disponible es aún limitada y se encuentra distribuida en áreas afines con diversos enfoques. Por ello, este capítulo adopta presentar resultados relevante obtenidos mediante una Revisión Sistemática de la Literatura descrita en el Capítulo 4. Se realizo con un enfoque exploratorio orientado a identificar conceptos, tendencias, enfoques y mecanismos relevantes que permitan evaluar trabajos relacionados con el problema y área de dominio de esta misma investigación.

3.1. Conceptualización del Auto-Diseño

En la vasta literatura sobre sistemas auto-adaptativos y auto-gestionados con origen en [23], [24], [25] se encuentran diversos enfoques para cubrir parcialmente la necesidad de intervención humana. Lo que se considera auto-diseño busca la completa independencia del sistema en ese sentido. Sin lugar a dudas, el trabajo de Porter Et. Al. [26] representa uno de los acercamientos más directos al concepto. Los autores conciben el *Self-Designing Software* como un paradigma en el cual el propio software se convierte en un miembro activo de su equipo de desarrollo. Su técnica principal radica en el *Hot-Swapping* o intercambio en caliente, el cual permite sustituir componentes de software en un sistema en ejecución de manera segura mediante el uso de bloques de construcción (*building blocks*) validados y provistos de interfaces bien definidas.

Sin embargo, desde la perspectiva de la ingeniería de software clásica, este enfoque no satisface completamente el ciclo de diseño, ya que no concibe el diseño como el conjunto

de actividades estructuradas que entregan artefactos de diseño formales (como diagramas arquitectónicos, especificaciones de componentes, interfaces, etc.) [6], si no un reemplazo de componentes establecidos dependiendo de la funcionalidad.

Con una especificación mas concreta centrada en estructuras de datos, el trabajo [27] describe una metodología para automatizar el diseño de estructuras de almacenamiento personalizadas. En lugar de depender de diseños tradicionales e inflexibles, este enfoque construye estructuras a la medida partiendo de un conjunto de primitivas de diseño lógicas y físicas clasificadas en su "Tabla Periódica de las Estructuras de Datos"[28]. El conjunto combinatorio de estas primitivas conforma un vasto espacio de diseño en el cual se evalúan rigurosamente los compromisos entre rendimiento y utilización de recursos. Tras este análisis, el candidato óptimo es enviado a un generador de código que sintetiza el diseño en código C++ ejecutable para el sistema final.

De manera complementaria en esta especificación, la herramienta COZY desarrollada por Loncaric et al. [29], [30] presenta un marco de síntesis lógica de colecciones de datos. La entrada del sistema consiste en una especificación declarativa de alto nivel que describe el estado abstracto (datos lógicos como usuarios, grupos o membresías), las consultas requeridas y las operaciones de actualización. A través de un bucle iterativo de refinamiento, incrementalización y verificación formal apoyado por el resolvedor SMT Z3 (mediante síntesis inductiva guiada por contra ejemplos, CEGIS), COZY genera estructuras de datos intrusivas y seguras por diseño, listas para su compilación.

3.2. Automatización del Diseño

La automatización de las tareas de diseño se ha abordado desde múltiples paradigmas de cómputo, respondiendo a la complejidad del dominio y al nivel de abstracción objetivo.

3.2.1. Auto-diseño basado en Inteligencia Artificial y Aprendizaje

En la generación de lógica y algoritmos, [31] proponen el marco ALDes (*Autoregressive Learning-based Designer*), el cual formula el diseño de algoritmos metaheurísticos como una tarea de generación de secuencias. A diferencia de los métodos tradicionales con estructuras fijas, ALDes utiliza redes autorregresivas basadas en Transformers para inferir secuencias con estructuras de control arbitrarias (bucles, condicionales) y longitudes variables, permitiendo la síntesis autónoma de algoritmos adaptados a problemas específicos. Por su parte, Yi y Qu [32] exploran el uso de Aprendizaje por Refuerzo (Proximal Polici

Optimisation - PPO) sobre un espacio de características dependientes de la instancia y del estado de búsqueda, logrando la combinación dinámica de componentes algorítmicos.

3.2.2. Autodiseño bio-inspirado, de reúso y búsqueda

Dentro de la Ingeniería de Software Basada en Búsqueda (SBSE), Ramírez et al. [33] presentan un enfoque evolutivo para el descubrimiento e inferencia autónoma de arquitecturas de software a partir de diagramas de clases. La aproximación utiliza algoritmos genéticos guiados por métricas de cohesión y acoplamiento para agrupar clases en componentes y descubrir interfaces arquitectónicas. En el ámbito de la refactorización estructural, Houichime y El Amrani [34] proponen el marco ODR, el cual codifica las propiedades del software mediante números complejos en un espacio vectorial unificado, permitiendo aplicar heurísticas de optimización para refactorizar y mejorar la calidad del diseño de forma directa.

3.2.3. Autodiseño basado en modelado de agentes

La emergencia de los Grandes Modelos de Lenguaje (LLMs) ha impulsado el desarrollo de sistemas multi-agente para la ingeniería de requerimientos y el diseño arquitectónico. Dam et al. [35] introducen un marco conceptual multi-agente donde agentes autónomos interpretan requerimientos en lenguaje natural, generan modelos UML y colaboran en la resolución de conflictos semánticos y fusión de versiones.

Asimismo, Zhang et al. [36] proponen el marco MAAD (*Multi-Agent Architecture Design*), en el cual agentes especializados (Analista, Modelador, Diseñador y Evaluador) simulan el flujo de trabajo de un equipo humano para traducir especificaciones SRS en planos arquitectónicos verificados sin incurrir en alucinaciones típicas de esquemas *zero-shot*. De igual forma, Rajendran et al. [37] diseñan un entorno multi-agente colaborativo y competitivo que emplea protocolos de subasta y consenso para balancear atributos de calidad en conflicto (rendimiento, seguridad y mantenibilidad) durante la refactorización arquitectónica.

3.3. Conceptos Emergentes y Mecanismos de Control en Autodiseño

El auto-diseño requiere motores de ejecución capaces de efectuar cambios en tiempo de ejecución de manera segura y controlada. Affonso et al. [38] presentan la arquitectura de referencia RA4SaS, basada en reflexión computacional, la cual permite inspeccionar, desensamblar y reconstruir entidades orientadas a objetos en tiempo de ejecución mediante metadatos y plantillas.

Por otro lado, Weyns y Andersson [39] formalizan la frontera entre la auto-adaptación convencional y la auto-evolución basándose en el concepto de Dominio de Diseño Operacional (ODD). Los autores demuestran que cuando un sistema enfrenta requerimientos o entornos no previstos en su ODD inicial ($ODD_{Se} = ODD_{Si} \cup ODD_e$), la simple adaptación de parámetros es insuficiente, obligando al sistema a evolucionar su propia estructura de manera autónoma.

3.4. Comparación de Trabajos

A continuación, la Tabla 3.1 sintetiza y clasifica las contribuciones clave revisadas en la literatura según su nivel de abstracción, técnica empleada, fase de aplicación y el artefacto generado.

Tabla 3.1: Matriz comparativa de enfoques de automatización y auto-diseño de software.

Enfoque / Autor	**Nivel de Abstracción**	**Técnica Principal**	**Fase de Aplicación**	**Artefacto Generado**
Porter et al. [26]	Componentes / Módulos	Hot-Swapping & Meta-composers	Tiempo de Ejecución	Ensamblado dinámico
Idreos et al. [27]	Estructuras de Datos	Primitivas de Layout & Cost Models	Diseño / Auto-tuning	Código C++ optimizado
Loncaric et al. [29]	Estructuras de Datos	Síntesis Lógica & CEGIS (Z3)	Tiempo de Diseño	Estructuras intrusivas C++
Zhao et al. [31]	Algoritmos / Heurísticas	Transformers Autorregresivos	Tiempo de Diseño	Algoritmos de búsqueda
Ramírez et al. [33]	Arquitectura de Software	Algoritmos Evolutivos (SBSE)	Tiempo de Diseño	Componentes e Interfaces
Zhang et al. [36]	Arquitectura de Software	LLMs Multi-Agente & Consenso	Tiempo de Diseño	Diagramas UML / SRS
Rajendran et al. [37]	Arquitectura / Refactorización	Agentes Especializados & Subastas	Tiempo de Diseño / Runtime	Código Refactorizado
Affonso et al. [38]	Entidades de Software	Reflexión Computacional	Tiempo de Ejecución	Reconfiguración estructural
Weyns et al. [39]	Marco de Auto-Evolución	Análisis de ODD	Tiempo de Ejecución	Extensión del Dominio *ODD*

Capítulo 4

Propuesta

4.1. Metodología

La metodología consta de dos etapas principales, la primera que fue realizar una revisión sistemática de literatura sobre técnicas, marcos de trabajo y demás propuestas que llevaran a cabo el autodiseño, esta misma se llevo a cabo siguiendo la metodología prisma, los resultados obtenido ayudaron a fundamentar la toma de decision para construir la propuesta. La siguiente etapa fue la construcción de un sistema multiagentes, se compararon diferentes metodologías, pero se decanto por Prometheus ya que implementa los conceptos BDI de manera natural, la comparación con metodologías esta dada con (Gaia, Tropos, Mase, O-mase, Adelfe, Pass)

4.1.1. Revisión Sistemática de literatura

El primer paso para desarrollar y fundamentar mi propuesta fue realizar un revisión sistemática de literatura, en la que siguiendo los lineamientos PRISMA [40], con este proceso estructurado se realizo la revisión de manera exhaustiva, el objetivo principal de esta Systematic Literature Review (SLR), es conocer como se ha automatizado el diseño de sistemas, conocer el estado del arte permite reconocer vacíos en el mismo.

4.1.1.1. Formulación preguntas de investigación

La formulación de una pregunta o preguntas de investigación en cualquier investigación es la piedra angular de la misma, en una SLR no es la excepción, para esto se utilizo el marco de trabajo PICO [41], La Población (P), abarca los procesos de diseño de software,

desde las tareas tradicionales automatizadas, hasta el auto-diseño bajo el marco de los sistemas de auto-ingeniería (SES) o que tengan relación directa con ellos. La intervención (I) se refiere a todos los enfoques automatizados, autónomos o impulsados por IA dentro de la basta literatura, para apoyar a dichas actividades. Estos enfoques comprenden modelos, técnicas y mecanismos destinados a permitir que los sistemas generen, evalúen, o evoluciones artefactos de diseño de manera autónoma. La Comparación (C), implica un análisis comparativo entre las metodologías o técnicas convencionales, contra enfoque progresivamente automatizados. Finalmente, el resultado previsto, denominado Resultado (Outcome en ingles), destaca en una síntesis sistemática del conocimiento existente para mapear esta evolución tecnológica, evaluar las teorías e identificar las brechas de investigación pendientes que obstaculizan la realización de un verdadero auto-diseño.

Con base a esta formulación las siguientes preguntas son propuestas:

- **RQ1:** ¿De que maneras se han automatizado o hecho autónomos los procesos tradicionales de ingeniería de diseño de software?
- **RQ2:** ¿Que términos o conceptos emergentes son equivalentes o cercanos a una actividad de diseño totalmente autónoma?
- **RQ3:** ¿Como se ha abordado el **Autodiseño** de sistemas en el contexto de los SES?

4.1.1.2. Estrategia de búsqueda

Para contestar a la primer pregunta de investigación (RQ1), el análisis fue estructurado alrededor de las actividades en el diseño de software, considerando su correspondencia con enfoques automáticos o autonómico, las etapas se muestran en la tabla 4.1.

Tabla 4.1: Actividades de diseño y sus correspondientes artefactos de diseño.

Actividades de diseño	**Artefactos de diseño**
Diseño de Arquitectura	Arquitectura del sistema
Diseño de interfaz	Especificación de interfaces
Diseño de componentes	Especificación de componentes
Diseño de estructuras de datos	Especificación de estructuras de datos
Diseño de algoritmos	Especificación de algoritmos

Esta descomposición facilita evaluar en que medida cada fase del diseño ha sido reinterpretada, respaldada o automatizada bajo los principios de automatización de auto-gestión.

Esto ofrece una visión amplia del diseño en nuestra búsqueda. De tal manera que una búsqueda granular se consigue con sub-preguntas de investigación, al final se contesta la RQ1.

- **SRQ1.1** ¿De qué manera se ha conceptualizado, implementado o evaluado el autodiseño arquitectónico o la automatización del diseño arquitectónico en la literatura?
- **SRQ1.2** ¿Cómo se ha generado o abordado la noción de autodiseño de interfaces o de diseño automatizado de interfaces en la investigación existente?
- **SRQ1.3** ¿En qué medida se ha explorado y tratado el autodiseño de componentes o el diseño automatizado de componentes en estudios previos?
- **SRQ1.4** ¿Cómo se han definido e investigado en la literatura los enfoques sobre el autodiseño de estructuras de datos o el diseño automatizado de estructuras de datos?
- **SRQ1.5** ¿Cómo se ha abordado el autodiseño de algoritmos o el diseño automatizado de algoritmos?

Para proveer de una respuesta mas comprensiva a la segunda pregunta de investigación (RQ2), se realizo una búsqueda exploratoria de manera exploratoria, usando terminología reciente y emergente. Este paso apunta a identificar conceptos e investigaciones que de una manera no explicita busca el auto-diseño. como resultado tenemos tópicos como diseño evolucionario de sistemas de software, diseño generativo en software, automatización del diseño de software e inteligencia artificial para ingeniería de software (AI4SE), estos tópicos se añadieron a la estrategia de búsqueda. Por ultimo, la manera de abordar la tercer pregunta de investigación (RQ3), se adopto una estrategia dirigida por el concepto clave de auto-diseño, algo que de manera explicita nos dirigiera a esto. así que se suman los siguiente términos: Auto-diseño, Sistemas de Auto-Arquitectura y Sistemas autosuficientes.

Las búsquedas de la SLR fue realizada en cuatro bases de datos de alto impacto en el campo de la ingeniería de software: Scopus, Web of Science, IEEE Xplore y ACM Digital Library. Estas plataformas se seleccionaron debido a la disponibilidad de acceso institucional completo y a su amplia cobertura de publicaciones revisadas por pares, incluidos artículos de revistas, actas de conferencias y capítulos de libros relevantes para la inteligencia artificial, la ingeniería de software y los sistemas autónomos y autonómicos. Google Scholar si bien es un buen motor de búsqueda, no es una base de datos de manera directa y presenta limitaciones en la replicabilidad de las búsquedas, por lo tanto no fue una fuente primaria de búsqueda [42], sin embargo en el proceso de bola de nieve (snowballing) si se uso para la búsqueda de las citas, tanto de entrada como de salida.

Las estrategias de búsqueda se elaboraron sistemáticamente combinando términos relacionados con el tema mediante operadores booleanos (`AND`, `OR`) para estructurar explícitamente sus relaciones lógicas. La búsqueda se restringió al campo `TITLE-ABS-KEY` con el fin de aumentar la precisión conceptual y reducir el ruido en la recuperación de información. Posteriormente, se aplicaron criterios de inclusión y exclusión predefinidos para refinar y validar el conjunto de estudios resultante. Un ejemplo de una consulta completa esta escrita en la tabla 4.2.

Tabla 4.2: Ejemplo de una consulta de búsqueda completa (Scopus)

Database	Scopus
Search Field	TITLE-ABS-KEY
Time Range	2015–2026
Subject Areas	Computer Science (COMP), Engineering (ENGI)
Complete Query	`TITLE-ABS-KEY ((("self-architecting" OR "self architectural" OR " auto-architecting" OR "auto architectural") W/3 "system*") OR (("self-architectural" OR "auto-architectural") W/3 "design")) AND PUBYEAR > 2014 AND PUBYEAR < 2027 AND (LIMIT-TO (SUBJAREA, "COMP") OR LIMIT-TO (SUBJAREA, "ENGI"))`

Las consultas presentadas a continuación corresponden a un conjunto resumido de expresiones que inicialmente fueron ejecutadas en Scopus, y posteriormente adaptadas a las sintaxis especificas de las diferentes bases de datos, se listan las consultas en la tabla 4.3.

Tabla 4.3: Resumen de las consultas en Scopus (Obviando los filtros temporales y de dominio para mantener la brevedad).

Topic/filter name	**Summarized Query (Field: TITLE-ABS-KEY)**
1. Self-Architecting Systems	("self-architecting system*" OR "auto-architecting system*" OR "self-architectural design" OR "auto-architectural design")
2. Self-Design	("self-design" OR "self design" OR "auto-design" OR "auto design" OR "self software design" OR "auto software design")
3. Evolutionary Design in Software Systems	(("evolutionary design" OR "evolutionary software design") AND ("software system*" OR "software engineering"))
4. Generative Design SW Systems	(("generative design" OR "generative software design") AND ("software system*" OR "software engineering"))
5. Software Design Automation	("software design automation" OR "design automation in software engineering" OR "automated software design" OR "AI for software design")
6. Self-Sufficient Systems	("self-sufficient system*" OR "autonomous self-sufficient software" OR "self-sufficient software system*")
7. AI for Software Engineering	(("AI for software engineering" OR "artificial intelligence in software engineering" OR "machine learning in software engineering"))
8. Self/Auto-Architectural Design	("self-architectural design" OR "auto-architectural design" OR "self adaptive architecture" OR "self-adaptive design")
9. Self/Auto-Interface Design	("self-interface design" OR "auto-interface design" OR "autonomous interface design" OR "adaptive interface design")
10. Self/Auto-Component Design	("self-component design" OR "auto-component design" OR "autonomous component design" OR "self-generated component*")
11. Self/Auto-Data S. Design	("self-data structure design" OR "auto-data structure design" OR "autonomous data structure" OR "self-optimizing data structure*")
12. Self/Auto-Algorithm Design	("self-algorithm design" OR "auto-algorithm design" OR "autonomous algorithm" OR "automated algorithm" OR "generative algorithm design" OR "AI-based algorithm")

Las consultas fueron ejecutadas de manera independiente y posteriormente los resultados fueron unificados (eliminando duplicados). El conjunto final de artículos formaron los *estudios iniciales seleccionados* de esta revisión, recordando que fueron filtrados mediante los criterios de inclusión y exclusión descritos a continuación.

4.1.1.3. Criterios de Inclusión y de exclusión

Para garantizar la calidad y relevancia de los estudios seleccionados, se establecen criterios explícitos de inclusión y exclusión de acuerdo al objetivo de esta revisión, la cual se centra en la automatización del diseño en el marco de los SES.

Criterios de inclusión

- **CI1: Periodo Temporal.** publicaciones desde **2015 hasta Marzo de 2026**. Un rango inicial (2005 – 2025) fue considerado, pero se actualizo para buscar técnicas y avances en el área de la Inteligencia Artificial, evadiendo la literatura del *computo autonómico* para enfocarnos unicamente en los sistemas de autoingeniería.
- **CI2: Idioma.** Solo artículos en **Ingles** fueron incluidos para alinearnos con el lenguajes mas adoptado en la divulgación científica en el mundo.
- **CI3: Disponibilidad y Claridad.** los estudios deben proveer **acceso completo** y una **metodología claramente definida** para permitir un análisis comprensivo y total de los mismo.
- **CI4: Dominio especifico.** Unicamente investigaciones en el campo de la **Ingeniería de software** que aborde de manera explicita o de manera indirecta cubra lo que es el auto-diseño de una manera unificada como proceso total o parcial.
- **CI5: Relevancia Científica.** Selección de estudios que han demostrado impacto en la comunidad científica a través de las citas, mientras se busca balancear este criterio, evaluando publicaciones reciente que no tienen un acumulado de citas pero si un gran contribución teórica o técnica en el tema.

Criterios de exclusión

Los siguientes criterios fueron usados para filtrar artículos que no coincidan con los requerimientos de la investigación:

- **CE1: Duplicación. publicaciones duplicadas** o **múltiples versiones** del mismo estudio; en cuyos casos, solo el mas completo o versión mas reciente sera considerada.

- **CE2: Tipo de Documento. Documentos incompletos**, **resúmenes extendidos**, o **publicaciones de posters** que carecen de suficiente detalle técnico para su evaluación.
- **EC3: Irrelevancia Disciplinaria.** estudios fuera del dominio de **Ingeniería de Software** sin un enlace directo al auto-diseño.
- **CE4: Deficiencias Metodológicas.** estudios que carezcan de **rigor metodológico**, particularmente lo que no tienen una clara descripción de las técnicas propuestas, algoritmos o conceptos.

Estos filtros (revisar Tabla 4.4) permiten la identificación de estudios representativos y de alta calidad, centrados en avances recientes en el ámbito conceptual y prácticos en el área de auto-diseño y auto-ingeniería, garantizando así la validez y relevancia de esta revisión sistemática de la literatura (este capitulo en particular).

Tabla 4.4: Resumen de los criterios de inclusión y exclusión

Tipo	**Criterio**
Inclusión	**CI1:** Publicaciones entre 2015 y 2025.
	CI2: Publicaciones en Ingles.
	CI3: Disponibilidad completa con metodología detallada.
	CI4: Centrado unicamente en software engineering, self-engineering, or self-design.
	CI5: Estudios relevantes con contribuciones recientes o numero de citas destacables.
Exclusión	**CE1:** Publicaciones duplicadas o múltiples versiones del mismo estudio.
	CE2: Resúmenes, posters o documentos incompletos.
	CE3: Estudios fuera del dominio de los sistemas de software.
	CE4: Trabajos con falta de detalles en su metodología o rigor científico.

4.1.1.4. Proceso de selección de estudios

Este bloque describe como fue la selección de estudios siguiendo las pautas de PRISMA. La Figura 4.1 ilustra el flujo general de identificación de estudios, cribado (aplicación de criterios de inclusión y exclusión), evaluación de elegibilidad (lectura del resumen y, posteriormente, del documento completo) e inclusión final. Los artículos semilla (conjunto inicial) se seleccionaron a partir de los estudios que cumplían los criterios de inclusión y obtuvieron las puntuaciones más altas en la evaluación de calidad durante la búsqueda inicial en bases de datos. Posteriormente, se identificaron estudios adicionales mediante la técnica de bola de nieve (hacia atrás y hacia adelante), siguiendo las directrices propuestas por [43]. Este proceso iterativo consistió en examinar las listas de referencias de los artículos semilla (bola de nieve hacia atrás) e identificar los artículos que los citaban (bola de nieve hacia adelante), aplicando los mismos criterios de inclusión y exclusión en cada iteración.

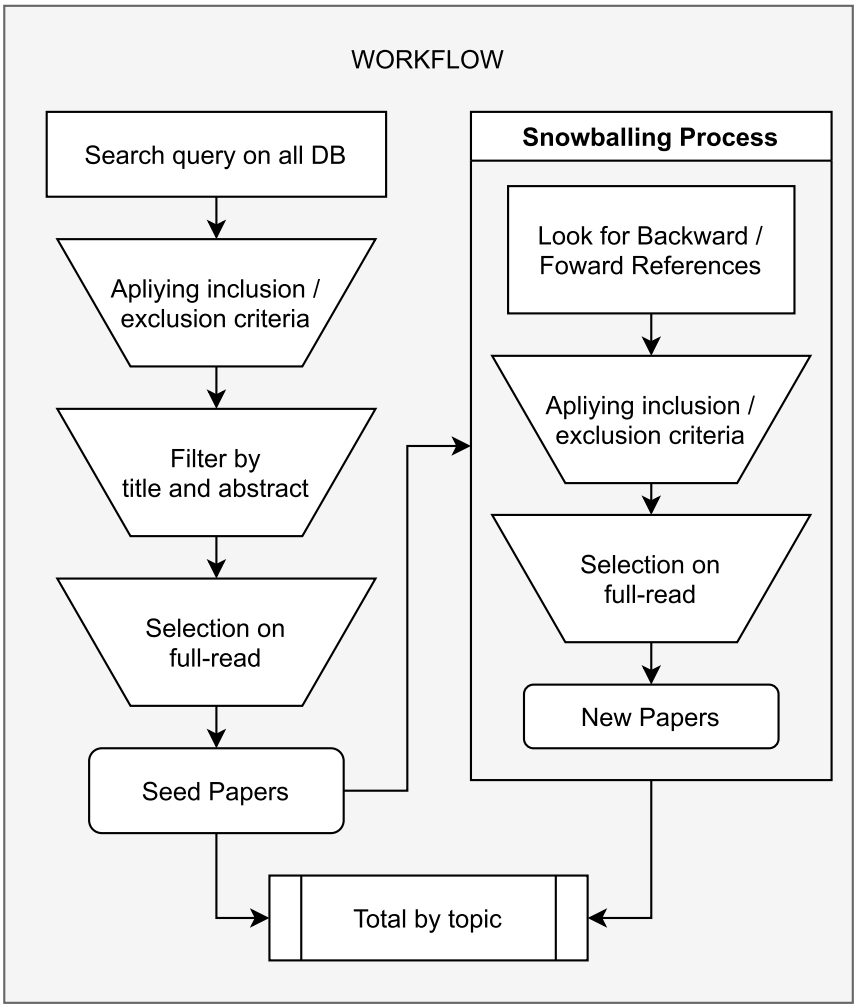


Figura 4.1: Revisión del flujo de trabajo basado en PRISMA.(Diagrama elaborado en Draw.io)

Para complementar el flujo de PRISMA, se añade una vista preliminar a manera de cuantificar los resultados, así detallar los los estudios seleccionados por cada etapa del proceso y especificando el tópico en particular. Esta representación permite comprender mejor cómo el conjunto inicial de estudios recuperados se refinó progresivamente hasta llegar a los estudios seleccionados finalmente y analizados en esta revisión. La Figura 4.2 resume este proceso mediante la especificación explicita del umero de artículos por cada etapa de selección.

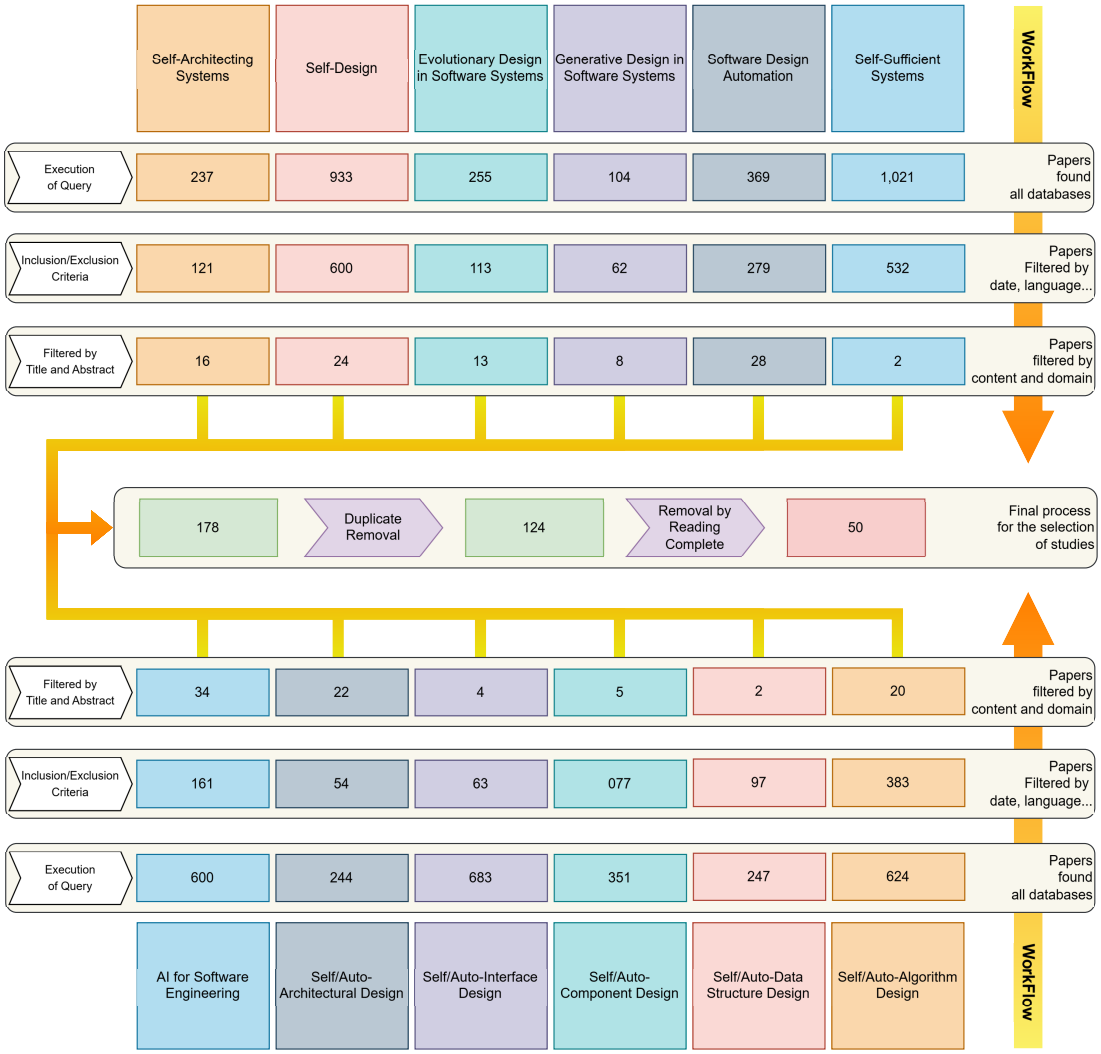


Figura 4.2: Proceso de selección de artículos con resultados cuantitativos por etapa de cribado. (Diagrama elaborado en Draw.io)

4.1.1.5. Extracción de datos

La extracción de datos se realizó exclusivamente a partir de los estudios que cumplían con los criterios de inclusión definidos y presentados en la Tabla 4.4. Este proceso se llevó a cabo de manera sistemática para garantizar la coherencia y la trazabilidad de la información recopilada de cada estudio seleccionado. Dada la heterogeneidad de la literatura analizada, se empleó un análisis narrativo para sintetizar la información. En este contexto, los enfoques analizados se organizaron en categorías temáticas derivadas de los objetivos de la revisión y de las preguntas de investigación formuladas. Esta estrategia permitió identificar patrones recurrentes, tendencias emergentes y áreas poco exploradas dentro de

los estudios seleccionados. Asimismo, la estructuración temática facilitó una evaluación comparativa de cómo cada estudio aborda el autodiseño en el contexto de la ingeniería de software, destacando las similitudes y diferencias entre los enfoques propuestos, así como el grado en que sus hallazgos contribuyen a responder a las preguntas de investigación planteadas en esta revisión.

4.1.1.6. Resultados

Los resultados completos están en el articulo titulado "Automated Software Design Towards Zero Human Intervention: A Systematic Literature Review" que actualmente esta sometido (en proceso de validación), pero para este trabajo se extrajeron los resultados mas importantes en el capítulo 3. La siguiente es una síntesis de lo que fue la discusión.

4.1.1.7. Discusión

La revisión revela un punto de inflexión hacia la automatización del diseño de software a través de cuatro paradigmas principales: la refactorización basada en búsqueda (SBR), el autodiseño algorítmico mediante mejora genética (GI), la adaptabilidad formal con ontologías y planificación (PDDL), y la transición hacia la Ingeniería de Software 3.0 (SE 3.0) basada en ecosistemas multiagente. Aunque estos enfoques permiten manipular arquitecturas complejas y redefinen el rol del desarrollador hacia el de un orquestador estratégico, enfrentan importantes desafíos técnicos: la pérdida de semántica en los modelos matemáticos, la inestabilidad durante el reemplazo de código en caliente, la sobrecarga computacional en entornos inciertos y el riesgo de alucinaciones de diseño ante la falta de restricciones verificables y gobernanza en los agentes. Finalmente, se evidencia una brecha crítica entre el potencial teórico y la práctica, ya que la mayoría de los estudios se limitan a entornos controlados o sintéticos; por ello, se concluye que la integración del razonamiento simbólico con modelos de lenguaje es indispensable para lograr arquitecturas autogeneradas transparentes, funcionales y viables en entornos de producción reales.

4.1.2. Propuesta de un Sistema Multi-Agentes (MAS) para satisfacer el auto-diseño

El auto-diseño se define como el conjunto de actividades que un sistema realiza para producir artefactos, especificaciones y demás elementos necesarios de diseño (diagramas por ejemplo),para establecer una representación del software que deberá ser construido a

partir de un conjunto de requerimientos. Estas actividades pueden ocurrir tanto durante procesos de evolución o metamorfosis del sistema hacia una nueva configuración arquitectónica (nuevo diseño de software).

Para automatizar estas actividades se propone la utilización de un sistema multiagente (MAS), en el cual diferentes agentes especializados participan en las distintas actividades que conforman el proceso de diseño. Cada agente posee una responsabilidad específica, un conjunto de conocimientos y mecanismos para tomar decisiones sobre los artefactos bajo su responsabilidad.

La utilización de múltiples agentes permite distribuir el proceso de diseño en diferentes responsabilidades, evitando concentrar todas las decisiones en un único componente. Asimismo, permite establecer mecanismos de comunicación y coordinación mediante los cuales las decisiones tomadas por un agente pueden ser utilizadas como información por los demás agentes.

En este trabajo, el sistema multi-agente se organiza alrededor de cuatro responsabilidades principales: diseño arquitectónico, diseño de componentes e interfaces, selección de estructuras de datos y selección de algoritmos. Estas responsabilidades corresponden a diferentes actividades del proceso de diseño y mantienen dependencias entre sí.

El proceso tradicional puede representarse como:

$$\text{Requerimientos} \rightarrow \text{Arquitectura} \rightarrow \text{Componentes} \rightarrow \text{Interfaces} \rightarrow \text{Estructuras de Datos} \rightarrow \text{Algoritmos}$$

Este flujo representa la secuencia principal de diseño; sin embargo, no implica que las decisiones sean irreversibles. Una modificación realizada en una etapa posterior puede generar restricciones o incompatibilidades que requieran la reconsideración de decisiones previamente adoptadas. Por este motivo, la arquitectura propuesta contempla mecanismos de retroalimentación y coordinación entre agentes.

4.1.3. Desarrollo de la metodología Prometheus

Para especificar el sistema multiagente se empleó la metodología Prometheus, debido a que proporciona un proceso sistemático para identificar los agentes, sus objetivos, percepciones, acciones, capacidades y mecanismos de interacción [44].

Prometheus permite describir el sistema desde diferentes niveles de abstracción mediante tres fases principales: especificación del sistema, diseño arquitectónico y diseño detallado. Estas fases permiten pasar de la identificación de las necesidades generales del

sistema a la definición de la organización de los agentes y, posteriormente, a la especificación de su funcionamiento interno.

En el contexto de este trabajo, Prometheus se utiliza no solamente para representar un flujo de trabajo, sino para establecer las características que permiten considerar a los participantes del proceso de diseño como agentes. En particular, se consideran sus objetivos, percepciones, acciones, conocimientos y mecanismos de decisión.

Aunque idealmente todos los agentes podrían contar con una descripción detallada de su funcionamiento interno, el alcance de este trabajo se concentra en proporcionar una descripción de caja blanca del agente encargado de la selección de algoritmos. Este agente incorpora un mecanismo explícito de búsqueda mediante Monte Carlo Tree Search (MCTS) sobre una ontología de conocimiento. Los demás agentes son descritos principalmente desde su comportamiento externo y sus responsabilidades dentro del proceso de diseño.

4.1.3.1. Especificación del sistema

La fase de especificación del sistema tiene como propósito identificar las funcionalidades principales, los objetivos generales, las percepciones y las acciones que permiten definir el comportamiento esperado del sistema multiagente.

En esta fase se identifica el cambio en los requerimientos como uno de los principales estímulos del sistema. A partir de este cambio, los agentes deben analizar el diseño existente y determinar las modificaciones necesarias para satisfacer el nuevo estado de requerimientos.

Los objetivos generales identificados son:

- Analizar a que nivel los nuevos requerimientos modifican el diseño actual;
- Determinar los artefactos de diseño afectados;
- Generar o modificar los artefactos de diseño;
- Mantener la consistencia entre las diferentes decisiones de diseño;
- Seleccionar alternativas de diseño compatibles con los requerimientos;
- Coordinar las decisiones realizadas por los diferentes agentes.

A partir de estos objetivos se identifican cuatro agentes principales:

1. **Agente de arquitectura**: responsable de analizar los requerimientos y la arquitectura existente para determinar si debe realizarse una modificación incremental o una metamorfosis del sistema.

2. **Agente de componentes e interfaces**: responsable de determinar y adaptar los componentes y mecanismos de interacción necesarios para implementar la arquitectura seleccionada.

3. **Agente de estructuras de datos**: responsable de analizar las necesidades de representación y acceso a la información y seleccionar las estructuras de datos correspondientes.

4. **Agente de algoritmos**: responsable de caracterizar los problemas algorítmicos y seleccionar algoritmos que satisfagan las características y restricciones identificadas mediante un mecanismo de búsqueda basado en MCTS.

Cada agente recibe información procedente del entorno y de los demás agentes. Esta información constituye sus percepciones y permite actualizar el conocimiento utilizado para tomar decisiones.

4.1.3.2. Diseño arquitectónico

La fase de diseño arquitectónico establece la organización de los agentes y las relaciones de comunicación existentes entre ellos.

La arquitectura propuesta se organiza alrededor de un flujo principal de diseño:

$$IN_{\text{requerimientos}} \rightarrow Agent_{\text{arq}} \rightarrow Agent_{\text{comp/int}} \rightarrow Agent_{\text{est.datos}} \rightarrow Agent_{\text{alg}} \rightarrow OUT_{SDD}$$

donde $IN_{\text{requerimientos}}$ representa la entrada de requerimientos, mientras que $Agent_{\text{arq}}$, $Agent_{\text{comp/int}}$, $Agent_{\text{est.datos}}$, $Agent_{\text{alg}}$ representan los agentes de arquitectura, componentes e interfaces, estructuras de datos y algoritmos, respectivamente. Por ultimo OUT_{SDD} representa la salida como si fuera un SDD (software design document) con las especificaciones desarrolladas.

El flujo anterior representa la dependencia natural entre las decisiones. La arquitectura determina los componentes necesarios; los componentes e interfaces determinan las necesidades de comunicación; las estructuras de datos determinan las formas de representación y acceso a la información; y estas decisiones establecen restricciones que pueden ser utilizadas durante la selección de algoritmos.

Sin embargo, la interacción no se limita a una comunicación unidireccional. Cuando una decisión posterior resulta incompatible con decisiones previamente adoptadas, el agente puede generar una notificación o solicitud de reconsideración dirigida al agente responsable de la decisión afectada.

De esta manera, el sistema combina un flujo principal secuencial con mecanismos de retroalimentación:

$$A_i \rightarrow A_{i+1} \tag{4.1}$$

y

$$A_{i+1} \rightarrow A_i \tag{4.2}$$

cuando una decisión posterior requiere reconsiderar una decisión previa.

Este mecanismo permite representar el proceso de diseño como una actividad iterativa y no únicamente como una cadena rígida de procesamiento.

4.1.3.3. Diseño detallado

La fase de diseño detallado especifica el funcionamiento interno de los agentes. En esta etapa se definen sus metas, creencias, percepciones, acciones, capacidades e intenciones, siguiendo una aproximación basada en el modelo Belief-Desire-Intention (BDI).

Para cada agente se identifican:

- **Creencias**: Información que el agente mantiene acerca del sistema, los requerimientos y los artefactos de diseño.
- **Metas**: Estados que el agente intenta alcanzar.
- **Intenciones**: Decisiones adoptadas por el agente respecto de las acciones que ejecutará para alcanzar sus metas.
- **Percepciones**: Información obtenida del entorno o de otros agentes.
- **Acciones**: Operaciones que el agente puede ejecutar sobre los artefactos de diseño.
- **Capacidades**: Mecanismos mediante los cuales el agente puede realizar sus acciones.

El ciclo de operación de cada agente puede expresarse de manera general como:

Percepcion → Actualizacion de Creencias → Evaluacion de Metas → Intencion → Accion

Después de ejecutar una acción, el agente recibe nueva información y actualiza nuevamente sus creencias. Este ciclo permite incorporar comportamiento reactivo y mecanismos de reconsideración de decisiones.

El agente de algoritmos constituye el caso de estudio de caja blanca de esta investigación. Su funcionamiento interno incorpora una caracterización del problema algorítmico, una búsqueda sobre el espacio de soluciones representado mediante una ontología y un mecanismo MCTS para determinar la alternativa que mejor satisface las características del problema.

4.1.4. Análisis y validación del modelo de agentes

Una vez definidos los agentes y sus interacciones, se analiza si cada participante satisface las características necesarias para ser considerado un agente dentro del sistema multiagente.

Para este propósito se consideran las siguientes propiedades:

- autonomía;
- comportamiento reactivo;
- orientación hacia metas;
- capacidad de interacción;
- estado y conocimiento interno;
- capacidad de tomar decisiones;
- capacidad de modificar su comportamiento en función del contexto.

La autonomía se analiza verificando que el agente pueda determinar una acción a partir de sus metas, creencias y percepciones, sin que cada decisión deba ser especificada externamente.

La reactividad se analiza verificando que el agente pueda responder ante cambios en el entorno, modificaciones de requerimientos o información proporcionada por otros agentes.

La orientación hacia metas se verifica mediante la existencia de objetivos explícitos que guían la selección de acciones.

Finalmente, la capacidad de interacción se analiza mediante los mecanismos utilizados para intercambiar información, propuestas, restricciones y solicitudes de reconsideración entre los agentes.

Este análisis permite distinguir entre un componente que únicamente ejecuta una función predeterminada y un agente que mantiene un proceso interno de percepción, deliberación y acción.

4.2. Propuesta de marco conceptual y arquitectura

La propuesta integra los conceptos de auto-diseño, sistemas multiagentes, agentes BDI, coordinación entre agentes y mecanismos especializados de búsqueda.

El marco conceptual establece que el proceso de auto-diseño puede ser descompuesto en actividades de diseño especializadas, cada una de las cuales es responsabilidad de un agente. La especialización permite que cada agente utilice mecanismos de razonamiento apropiados para el tipo de decisión que debe realizar.

La arquitectura resultante se representa mediante cuatro agentes principales:

$$MAS = A_{arch}, A_{comp}, A_{ds}, A_{alg} \tag{4.3}$$

donde:

- A_{arch} representa el agente de arquitectura;
- A_{comp} representa el agente de componentes e interfaces;
- A_{ds} representa el agente de estructuras de datos;
- A_{alg} representa el agente de algoritmos.

Los agentes comparten un estado de diseño que permite mantener la trazabilidad entre las decisiones. Este estado contiene los artefactos generados y las restricciones derivadas de decisiones anteriores.

4.2.1. Definición conceptual y formal

Sea R el conjunto de requerimientos del sistema:

$$R = r_1, r_2, \ldots, r_n \tag{4.4}$$

y sea:

$$D = D_A, D_C, D_{DS}, D_{ALG} \tag{4.5}$$

el conjunto de artefactos de diseño correspondientes a arquitectura, componentes e interfaces, estructuras de datos y algoritmos.

Cada agente A_i recibe un estado de diseño S y un conjunto de percepciones P_i, actualiza sus creencias B_i y determina una intención I_i orientada a alcanzar una meta G_i.

Formalmente:

$$A_i : (B_i, P_i, G_i, S) \rightarrow (I_i, D_i) \tag{4.6}$$

donde D_i representa la decisión o artefacto producido por el agente.

El diseño global puede expresarse como:

$$D^* = D_A \cup D_C \cup D_{DS} \cup D_{ALG} \tag{4.7}$$

sujeto a las restricciones establecidas por los requerimientos y por las dependencias existentes entre los diferentes artefactos.

4.2.2. Propuesta detallada

4.2.2.1. Agente de arquitectura

El agente de arquitectura analiza los cambios en los requerimientos y determina el impacto que estos producen sobre la arquitectura existente. A partir de sus creencias acerca del sistema, puede adoptar como intención modificar la arquitectura existente o iniciar una metamorfosis hacia una nueva arquitectura.

Sus principales responsabilidades son:

- analizar cambios en los requerimientos;
- evaluar la arquitectura existente;

- identificar elementos arquitectónicos afectados;
- modificar la arquitectura cuando sea suficiente una adaptación;
- generar una nueva arquitectura cuando las modificaciones requeridas superen las capacidades de adaptación.

4.2.2.2. Agente de componentes e interfaces

Este agente analiza la arquitectura y determina los componentes e interfaces necesarios para implementarla. Además, considera las condiciones operativas del sistema para determinar si las configuraciones actuales continúan siendo adecuadas.

Entre sus responsabilidades se encuentran:

- seleccionar componentes;
- configurar relaciones entre componentes;
- adaptar interfaces;
- detectar incompatibilidades;
- modificar mecanismos de comunicación;
- adaptar configuraciones ante cambios del entorno.

Un ejemplo corresponde a la detección de una modificación en las condiciones de carga del sistema. El agente puede determinar que la configuración actual de componentes e interfaces no satisface los requerimientos y adoptar una intención de reconfiguración, por ejemplo mediante la incorporación o modificación de un mecanismo de balanceo de carga.

4.2.2.3. Agente de estructuras de datos

El agente de estructuras de datos analiza los requerimientos de almacenamiento y acceso a información junto con los artefactos generados por los agentes anteriores.

Sus decisiones consideran, entre otros factores:

- operaciones requeridas;
- patrones de acceso;

- volumen de información;
- restricciones de memoria;
- restricciones temporales;
- relaciones con componentes e interfaces.

A partir de esta información, el agente selecciona, modifica o reemplaza estructuras de datos para satisfacer las restricciones establecidas.

4.2.2.4. Agente de algoritmos

El agente de algoritmos recibe los requerimientos algorítmicos y los artefactos generados por los agentes anteriores. Su primera actividad consiste en caracterizar el problema mediante un conjunto de propiedades que permiten identificar el espacio de soluciones relevante.

La caracterización del problema se representa mediante:

$$P = (Objective, Interaction, Knowledge, Processing, Layer, Uncertainty) \tag{4.8}$$

A partir de esta representación, el agente explora una ontología de algoritmos mediante Monte Carlo Tree Search. Las expansiones del árbol corresponden a restricciones o características que permiten reducir el espacio de soluciones hasta identificar alternativas compatibles con el problema.

El mecanismo de búsqueda constituye el comportamiento de caja blanca del agente de algoritmos, permitiendo analizar explícitamente cómo se construye el espacio de búsqueda, cómo se evalúan las alternativas y cómo se actualizan los valores asociados a las ramas exploradas.

4.2.3. Mecanismo de coordinación y consenso

Debido a que las decisiones de diseño presentan dependencias entre sí, los agentes requieren un mecanismo para detectar y resolver inconsistencias.

Se propone un proceso iterativo de coordinación compuesto por cuatro etapas:

1. generación de propuestas;

2. evaluación de compatibilidad;

3. identificación y negociación de conflictos;

4. aceptación o reconsideración de las decisiones.

Cada propuesta contiene una decisión, las restricciones satisfechas, las dependencias identificadas y una justificación de la decisión.

Cuando dos decisiones son compatibles, ambas pueden incorporarse al estado global del diseño. Cuando existe un conflicto, los agentes involucrados reconsideran sus decisiones utilizando la nueva información disponible.

El proceso puede expresarse como:

$$Proponer \rightarrow Evaluar \rightarrow Detectar\ conflicto \rightarrow Reconsiderar \rightarrow Aceptar \tag{4.9}$$

Este mecanismo permite mantener la autonomía de los agentes sin convertir la coordinación en una decisión centralizada. Cada agente conserva la responsabilidad sobre su dominio de diseño, mientras que el estado compartido permite identificar las dependencias entre las decisiones.

4.2.4. Resumen conceptual

El sistema se estructura como un sistema multiagente orientado al proceso de diseño de software. Los agentes colaboran mediante un flujo de trabajo secuencial, en el cual cada agente posee una responsabilidad de diseño específica y produce información que condiciona las decisiones de los agentes posteriores.

La metodología Prometheus se emplea para identificar y modelar los agentes, sus metas, percepciones, acciones, capacidades e interacciones. El modelo BDI proporciona una estructura para representar el conocimiento, las metas y las intenciones de los agentes, mientras que los mecanismos de coordinación permiten gestionar las dependencias y conflictos entre decisiones.

El flujo principal está constituido por los agentes de arquitectura, componentes e interfaces, estructuras de datos y algoritmos. Este flujo no se considera estrictamente lineal, debido a que una decisión puede provocar la reconsideración de decisiones previamente adoptadas.

Finalmente, el agente de algoritmos incorpora un mecanismo explícito de razonamiento mediante la caracterización del problema y la búsqueda MCTS sobre una ontología. De esta manera, el sistema combina una organización multiagente, comportamiento reactivo, toma de decisiones orientada a metas y mecanismos especializados de búsqueda para automatizar diferentes actividades del proceso de diseño de software.

Capítulo 5

Experimentación y Análisis de Resultados

5.1. Análisis de selección automática de algoritmos

5.1.1. Análisis de la arquitectura: auto-diseño

5.2. Experimentación

5.2.1. Caso de estudio: nuevo problema, nueva solución

5.2.2. Experimento: Identificación del Problema

5.2.3. Experimento: Identificación del Algoritmo

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones

6.2. Objetivos logrados

6.3. Trabajo a futuro

Referencias Bibliográficas

[1] IEEE Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK)*, 4.ª ed. Los Alamitos, CA: IEEE, 2025, ISBN: 978-0-7695-5166-1. dirección: https://www.computer.org/education/bodies-of-knowledge/software-engineering

[2] A. Ampatzoglou, A. Ampatzoglou, A. Chatzigeorgiou y P. Avgeriou, «The financial aspect of managing technical debt: A systematic literature review,» *Information and Software Technology*, vol. 64, págs. 52-73, 2015, ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2015.04.001 dirección: https://www.sciencedirect.com/science/article/pii/S0950584915000762

[3] W. K. G. Assunção, L. Marchezan, L. Arkoh, A. Egyed y R. Ramler, «Contemporary Software Modernization: Strategies, Driving Forces, and Research Opportunities,» *ACM Trans. Softw. Eng. Methodol.*, vol. 34, n.º 5, mayo de 2025, ISSN: 1049-331X. DOI: 10.1145/3708527 dirección: https://doi.org/10.1145/3708527

[4] G. P. Pérez Carrillo, «Nuevo Marco Conceptual de Sistemas de Autoingeniería y una Arquitectura para la Gestión de su Ciclo de Vida,» Tesis de Maestría en Ciencias, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional Unidad Guadalajara, Guadalajara, México, ago. de 2024.

[5] G. Pérez-Carrillo, M. Siller y L. Alonso, «Self-Engineering Systems: A New Conceptual Framework and a Reference Meta-Architecture for its Life Cycle Management,» *Automated Software Engineering*, mayo de 2026. DOI: 10.21203/rs.3.rs-9577141/v1 dirección: http://dx.doi.org/10.21203/rs.3.rs-9577141/v1

[6] I. Sommerville, *Software Engineering*, 9.ª ed. Boston: Addison-Wesley, 2011, ISBN: 978-0-13-703515-1.

[7] A. Smeda, M. Oussalah y T. Khammaci, «MADL: Meta Architecture Description Language,» en *Third ACIS Int'l Conference on Software Engineering Research, Management and Applications (SERA'05)*, 2005, págs. 152-159. DOI: 10.1109/SERA.2005.50

[8] P. B. Kruchten, «The 4+1 View Model of Architecture,» *IEEE Softw.*, vol. 12, págs. 42-50, 1995. DOI: 10.1109/52.469759

[9] I. Sommerville, *Software Engineering, Global Edition*, en, 10.ª ed. London, England: Pearson Education, jul. de 2015.

[10] «IEEE/EIA Guide - Industry Implementation of International Standard ISO/IEC 12207 : 1995 (ISO/IEC 12207) - Software Life Cycle Processes - Life Cycle Data,» *IEEE/EIA 12207.1-1997*, págs. 1-38, 1998. DOI: 10.1109/IEEESTD.1998.86377

[11] «IEEE Standard for Information Technology–Systems Design–Software Design Descriptions,» *IEEE STD 1016-2009*, págs. 1-35, 2009. DOI: 10.1109/IEEESTD.2009.5167255

[12] *Innovations in Multi-Agent Systems and Applications - 1*. Springer Berlin Heidelberg, 2010, ISBN: 9783642144356. DOI: 10.1007/978-3-642-14435-6 dirección: http://dx.doi.org/10.1007/978-3-642-14435-6

[13] Y. Zhang, M. Zhao, Y. Zhang e Y.-m. Cheung, «Trending Applications of Large Language Models: A User Perspective Survey,» *IEEE Transactions on Artificial Intelligence*, vol. 7, n.º 4, págs. 1835-1852, 2026. DOI: 10.1109/TAI.2025.3620272

[14] W. X. Zhao et al., «A Survey of Large Language Models,» *Frontiers of Computer Science*, vol. 20, n.º 12, mayo de 2026, ISSN: 2095-2236. DOI: 10.1007/s11704-026-60308-3 dirección: http://dx.doi.org/10.1007/s11704-026-60308-3

[15] T. Xiao y J. Zhu, *Foundations of Large Language Models*, 2026. arXiv: 2501.09223 [cs.CL]. dirección: https://arxiv.org/abs/2501.09223

[16] P. Shojaee, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio y M. Farajtabar, «The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity,» *arXiv preprint*, jun. de 2025. arXiv: 2506.06941 [cs.AI]. dirección: https://arxiv.org/abs/2506.06941

[17] M. Świechowski, K. Godlewski, B. Sawicki y J. Mańdziuk, «Monte Carlo Tree Search: a review of recent modifications and applications,» *Artificial Intelligence Review*, vol. 56, n.º 3, págs. 2497-2562, 2022, ISSN: 1573-7462. DOI: 10.1007/s10462-022-10228-y dirección: http://dx.doi.org/10.1007/s10462-022-10228-y

[18] T. R. Gruber, «A translation approach to portable ontology specifications,» *Knowledge Acquisition*, vol. 5, n.º 2, págs. 199-220, 1993, ISSN: 1042-8143. DOI: https://doi.org/10.1006/knac.1993.1008 dirección: https://www.sciencedirect.com/science/article/pii/S1042814383710083

[19] N. Guarino, D. Oberle y S. Staab, «What Is an Ontology?» En *Handbook on Ontologies*. Springer Berlin Heidelberg, 2009, págs. 1-17, ISBN: 9783540926733. DOI: 10.1007/978-3-540-92673-3_0 dirección: http://dx.doi.org/10.1007/978-3-540-92673-3_0

[20] I. Horrocks, P. F. Patel-Schneider y F. van Harmelen, «From SHIQ and RDF to OWL: the making of a Web Ontology Language,» *Journal of Web Semantics*, vol. 1, n.º 1, págs. 7-26, dic. de 2003, ISSN: 1570-8268. DOI: 10.1016/j.websem.2003.07.001 dirección: http://dx.doi.org/10.1016/j.websem.2003.07.001

[21] L. Duran Polanco, «Selección de algoritmos para la toma de decisiones en el IoT basada en un modelo semántico,» Tesis de Maestría, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional, Unidad Guadalajara, Guadalajara, Jalisco, México, feb. de 2026.

[22] L. Durán-Polanco y M. Siller, «A taxonomy for decision making in IoT systems,» *Internet of Things*, vol. 24, pág. 100 904, 2023, ISSN: 2542-6605. DOI: https://doi.org/10.1016/j.iot.2023.100904 dirección: https://www.sciencedirect.com/science/article/pii/S2542660523002275

[23] IBM, *Autonomic Computing: IBM's Perspective on the State of Information Technology*, White Paper, Armonk, NY, 2001. dirección: http://www.ibm.com/research/autonomic

[24] J. O. Kephart y D. M. Chess, «The vision of autonomic computing,» *Computer*, vol. 36, n.º 1, págs. 41-50, 2003.

[25] IBM Corporation, «Autonomic Computing: An Architectural Blueprint for Autonomic Computing,» IBM, White Paper, jun. de 2006.

[26] B. Porter, P. Faulkner Rainford y R. Rodrigues-Filho, «Self-Designing Software,» *Commun. ACM*, vol. 68, n.º 1, págs. 50-59, dic. de 2024, ISSN: 0001-0782. DOI: 10.1145/3678165 dirección: https://doi-org.access.biblioteca.cinvestav.mx/10.1145/3678165

[27] S. Idreos et al., «Learning Data Structure Alchemy,» eng, *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, vol. 42, págs. 46-57, 2019.

[28] S. Idreos et al., «The Periodic Table of Data Structures.,» *IEEE Data Eng. Bull.*, vol. 41, n.º 3, págs. 64-75, 2018.

[29] C. Loncaric, «Data structure synthesis,» en *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ép. FSE 2016, Seattle, WA, USA: Association for Computing Machinery, 2016, págs. 1073-1075, ISBN: 9781450342186. DOI: 10.1145/2950290.2983946 dirección: https://doi.org/10.1145/2950290.2983946

[30] C. Loncaric, M. D. Ernst y E. Torlak, «Generalized data structure synthesis,» en *Proceedings of the 40th International Conference on Software Engineering*, ép. ICSE '18, Gothenburg, Sweden: Association for Computing Machinery, 2018, págs. 958-968, ISBN: 9781450356381. DOI: 10.1145/3180155.3180211 dirección: https://doi.org/10.1145/3180155.3180211

[31] Q. Zhao, T. Liu, B. Yan, Q. Duan, J. Yang e Y. Shi, «Automated Metaheuristic Algorithm Design With Autoregressive Learning,» *IEEE Transactions on Evolutionary Computation*, vol. 29, n.º 5, págs. 2004-2018, oct. de 2025, ISSN: 1941-0026. DOI: 10.1109/tevc.2024.3464677 dirección: http://dx.doi.org/10.1109/TEVC.2024.3464677

[32] W. Yi, R. Qu y L. Jiao, «Automated algorithm design using proximal policy optimisation with identified features,» *Expert Systems with Applications*, vol. 216, pág. 119 461, abr. de 2023, ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.119461 dirección: http://dx.doi.org/10.1016/j.eswa.2022.119461

[33] A. Ramírez, J. R. Romero y S. Ventura, «An approach for the evolutionary discovery of software architectures,» *Information Sciences*, vol. 305, págs. 234-255, 2015, ISSN: 0020-0255. DOI: https://doi.org/10.1016/j.ins.2015.01.017 dirección: https://www.sciencedirect.com/science/article/pii/S0020025515000559

[34] T. Houichime e Y. El Amrani, «Optimized design refactoring (ODR): a generic framework for automated search-based refactoring to optimize object-oriented software architectures,» *Automated Software Engineering*, vol. 31, n.º 2, jun. de 2024, ISSN: 1573-7535. DOI: 10.1007/s10515-024-00446-9 dirección: http://dx.doi.org/10.1007/s10515-024-00446-9

[35] H. K. Dam, «Towards Multi-Agentic AI for automated software design and modelling: challenges and opportunities,» en *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, IEEE, nov. de 2025, págs. 311-314. DOI: 10.1109/asew67777.2025.00063 dirección: http://dx.doi.org/10.1109/ASEW67777.2025.00063

[36] R. Li et al., «Maad: Automate software architecture design through knowledge-driven multi-agent collaboration,» *arXiv preprint arXiv:2507.21382*, 2025.

[37] V. Rajendran, D. Besiahgari, S. C. Patil, M. Chandrashekaraiah y V. Challagulla, «A Multi-Agent LLM Environment for Software Design and Refactoring: A Conceptual Framework,» en *SoutheastCon 2025*, IEEE, mar. de 2025, págs. 488-493. DOI: 10.1109/southeastcon56624.2025.10971563 dirección: http://dx.doi.org/10.1109/SOUTHEASTCON56624.2025.10971563

[38] F. J. Affonso, G. Nagassaki Campos y G. Guiguer Menaldo, «A Reference Architecture Based on Reflection for Self-Adaptive Software: A Second Release,» *IEEE Access*, vol. 12, págs. 97 476-97 499, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3428368

[39] D. Weyns y J. Andersson, «From Self-Adaptation to Self-Evolution Leveraging the Operational Design Domain,» en *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, mayo de 2023, págs. 90-96. DOI: 10.1109/SEAMS59076.2023.00022

[40] M. J. Page et al., «The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,» *BMJ*, vol. 372, 2021. DOI: 10.1136/bmj.n71 eprint: https://www.bmj.com/content/372/bmj.n71.full.pdf. dirección: https://www.bmj.com/content/372/bmj.n71

[41] B. Kitchenham y S. Charters, «Guidelines for performing Systematic Literature Reviews in Software Engineering,» vol. 2, ene. de 2007.

[42] M. Gusenbauer y N. R. Haddaway, «Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources,» *Research Synthesis Methods*, vol. 11, n.º 2, págs. 181-217, ene. de 2020, ISSN: 1759-2887. DOI: 10.1002/jrsm.1378 dirección: http://dx.doi.org/10.1002/jrsm.1378

[43] C. Wohlin, «Guidelines for snowballing in systematic literature studies and a replication in software engineering,» en *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ép. EASE '14, ACM, mayo de 2014, págs. 1-10. DOI: 10.1145/2601248.2601268 dirección: http://dx.doi.org/10.1145/2601248.2601268

[44] L. Padgham y M. Winikoff, «Prometheus: A practical agent-oriented methodology,» en *Agent-oriented methodologies*, IGI Global Scientific Publishing, 2005, págs. 107-135.

Apéndice A

Código Fuente Relevante